OWASP
Smart Contract Security

scs.owasp.org ↗

Security Handbook :

DNS and Hosting

02

Security Handbook Series

GRANT SUPPORT • 2025-2026

ecosystem support program

Supported by Ethereum Foundation's Ecosystem Support Program (ESP)

Contents

Frontispiece

OWASP SMART CONTRACT SECURITY PROJECT

DNS and Hosting Security

About This Handbook

DNS and Hosting Security is part of the OWASP Smart Contract Security Handbook Series. The series extends smart contract security beyond contract code into the people, process, application, infrastructure, and operational layers surrounding Web3 systems.

Each handbook is designed as a defensive field reference for architects, engineers, security practitioners, incident responders, auditors, and organizational leaders. It complements the OWASP Smart Contract Security Verification Standard (SCSVS), Smart Contract Security Testing Guide (SCSTG), Smart Contract Weakness Enumeration (SCWE), Smart Contract Top 10, and SCS Checklist without replacing those resources.

SERIES EDITION **2026**

Copyright and License

Author and Creative Credits

PROJECT LEAD

Shashank | CredShields

CEO and Co-Founder

AUTHOR | PROJECT MAINTAINER

Pratik Lagaskar

Security Researcher

COVER PAGE DESIGNS

Siddharth Neekher

Lead UI Designer @ CredShields

OWASP Smart Contract Security Project

Purpose and Scope

This handbook covers attack playbooks and defensive guidance for the Domain Name System (DNS), traditional hosting, the InterPlanetary File System (IPFS), and the Ethereum Name Service (ENS) as they apply to Web3 front ends and resources. It works through DNS Security Extensions (DNSSEC), the chain of trust built from Delegation Signer (DS), Resource Record Signature (RRSIG), and DNSKEY records, and why decentralized protocols still depend on DNS at the point a user types a URL. It covers ENS's gasless DNSSEC integration and EIP-3668 CCIP Read (off-chain data lookup; not Chainlink's cross-chain protocol), guidance from the Internet Corporation for Assigned Names and Numbers (ICANN) and its Security and Stability Advisory Committee (SSAC) on DNS blocking and circumvention, and the trust assumptions behind IPFS gateways and pinning services.

A smart contract can pass every audit and still hand an attacker the front door, because most users reach a dApp through a domain name or an ENS name, not a contract address. DNS hijack, registrar compromise, ENS resolver takeover, and a poisoned IPFS gateway all sit upstream of the contract layer and defeat it without touching a single line of Solidity. This handbook treats that upstream layer, DNS, ENS, IPFS, and conventional hosting, as infrastructure a Web3 team must defend with the same rigor as its contracts, and it pairs each threat model with a playbook for the moment things go wrong.

Target Audience

Infrastructure and DevOps teams, security specialists, and protocol teams that rely on DNS, ENS, or IPFS to serve front ends and other Web3 resources.

How to Use This Handbook

Start with Part I to fix the architecture and the threat model: where DNS, ENS, and IPFS sit in a typical Web3 stack and what happens when each one fails. Parts II through V are organized by system, DNS, ENS, IPFS, and conventional hosting, and each pairs a threat model with hardening guidance and one or more incident playbooks a responder can follow directly. A team responding to a live domain hijack or a compromised ENS resolver should go straight to the relevant playbook in Part II, III, or IV rather than reading front to back. The appendices hold a glossary, a consolidated security checklist across all three systems, a playbook quick-reference table, and the full bibliography.

Relationship to SCSVS, SCSTG, and SCWE

This handbook sits outside the Smart Contract Security Verification Standard (SCSVS), the Smart Contract Security Testing Guide (SCSTG), and the Smart Contract Weakness Enumeration (SCWE) by design, because DNS, ENS resolution, and IPFS gateway trust are infrastructure risks that no amount of contract-level verification, testing, or weakness cataloging can close. The playbooks here give incident responders and infrastructure teams the DNS- and ENS-specific procedures that SCSVS assumes are already in place when it verifies a system's resolution and delivery integrity. It shares its infrastructure focus most closely with the Infrastructure Security Handbook (07), which covers the servers, cloud accounts, and network controls that DNS and hosting records point to, and it feeds directly into the Incident Response Handbook (06), whose general response process this handbook's playbooks specialize for domain, resolver, and gateway compromise.

Part I: Foundations

This part fixes the map that the rest of the handbook works from: where the Domain Name System (DNS), the Ethereum Name Service (ENS), and the InterPlanetary File System (IPFS) each sit in a typical Web3 stack, what happens to a user when one of them fails, and how this handbook's scope connects to the smart contract standards and to its closest sibling handbooks. Read it once before jumping to a system-specific Part or a playbook.

1. Introduction to DNS and Hosting Security in Web3

A user does not open a smart contract. A user opens a domain, or an ENS name, and trusts whatever that name resolves to. Every control this handbook covers, DNSSEC signatures, registrar locks, ENS resolver checks, IPFS content hashes, exists because that first act of resolution is a decision point an attacker can hijack without ever touching the protocol's Solidity or Move code. This chapter sets the vocabulary, surveys the attack classes with real incidents behind them, places the handbook against the OWASP Smart Contract Security (SCS) standards, and gives a reading path through the rest of the book.

1.1 DNS, ENS, and IPFS in Typical Web3 Architectures

Most Web3 products run on two parallel naming systems that a user rarely distinguishes between. The first is the classical DNS: a registrar holds the domain, a registry publishes it, and a set of authoritative nameservers answer queries that route a browser to a hosting provider or a content delivery network (CDN). DNS predates blockchains by decades, and RFC 1035 still defines the wire format every resolver on the internet speaks today. NIST SP 800-81 Rev. 3, *Secure Domain Name System (DNS) Deployment Guide*, published in March 2026 and superseding SP 800-81-2, is the current reference architecture for hardening authoritative and recursive DNS, encrypted DNS, protective DNS, DNS logging, and DNSSEC; Part 2 builds its guidance from that current baseline.

The second system is ENS, an on-chain naming layer that maps human-readable names like `curve.eth` to Ethereum addresses, content hashes, and other records through a resolver contract. ENS is not a replacement for DNS: it is a parallel registry that a growing share of wallets and

browsers resolve natively, and it can also import and re-publish ordinary DNS names through a DNSSEC-verified proof, a mechanism Part 3 covers in depth. Where DNS answers "what server do I connect to," ENS more often answers "what content hash or address does this human-readable name currently point to," and the two increasingly resolve into the same page load.

IPFS is the third leg, and it answers a different question entirely: not "where is this," but "what is this." Content on IPFS is addressed by a Content Identifier (CID), a value derived from a cryptographic hash of the content itself, by design, so that identical content always produces the same CID and any modification changes it. A front end can therefore be pinned to IPFS, referenced by its CID from an ENS content hash record, and served to a browser through any public gateway, with no single party controlling the bytes the user ultimately receives. The catch, covered in Part 4, is that gateways and pinning services are themselves points of trust: a CID guarantees the content is correct once retrieved, but says nothing about whether the gateway you queried is available, honest, or serving you a different, unpinned CID for a name you thought was fixed.

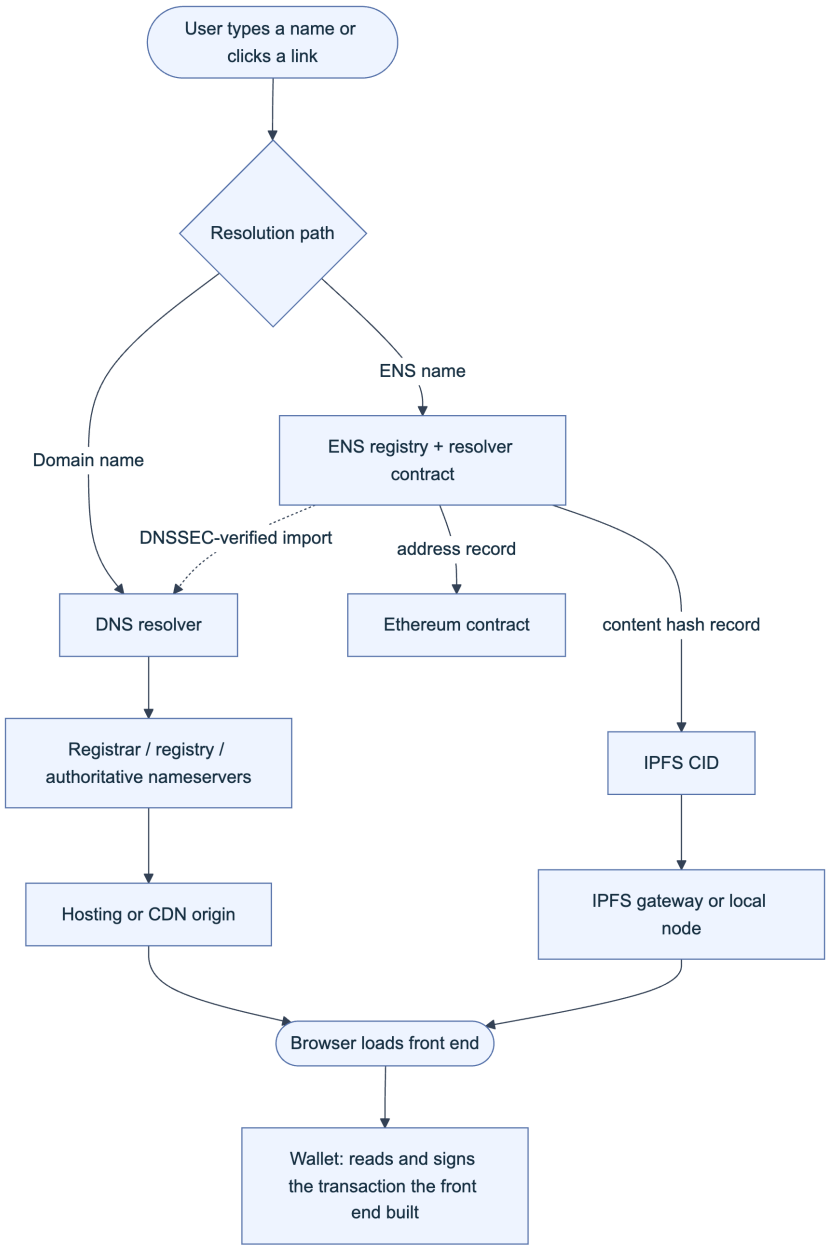

Figure 1. Two parallel naming paths converge on the same browser and the same wallet-signing step. A compromise anywhere upstream of the wallet, in DNS, in the ENS resolver, or in the IPFS gateway, delivers attacker-controlled content to the one place the user actually trusts.

The practical takeaway for an architect: know which path your users actually take. A team that assumes “we’re on IPFS, so we’re decentralized” while still pointing the primary marketing domain at a centralized DNS registrar with no registrar lock has not removed the DNS attack surface, it has only added IPFS’s own trust assumptions on top of it.

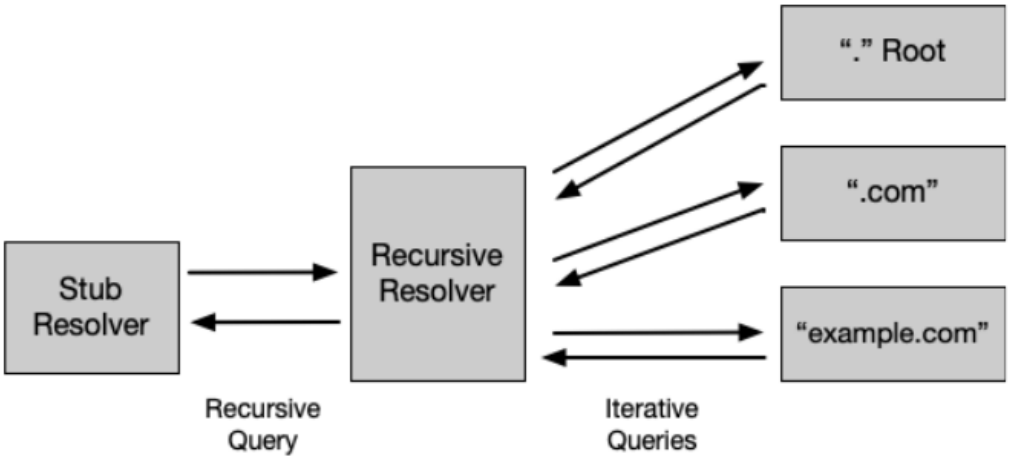

Fig. 4. DNS resolution

Figure. The resolver boundary that matters during triage: the client makes one recursive request, while the recursive resolver performs the iterative walk from the root to the authoritative zone. A forged referral, poisoned cache entry, or unauthorized authoritative answer changes what the client receives without changing its request. Source: NIST SP 800-81 Rev. 3, Figure 4, public domain (U.S. government work).

Encrypted DNS protects a different link than DNSSEC. DNS over HTTPS (DoH), DNS over TLS (DoT), and DNS over QUIC (DoQ) protect the channel between a client or forwarder and its recursive service; DNSSEC authenticates the data returned by the authoritative hierarchy. NIST's deployment model below also exposes a monitoring risk: unmanaged DoH clients can bypass an enterprise resolver unless policy routes them through the approved service.

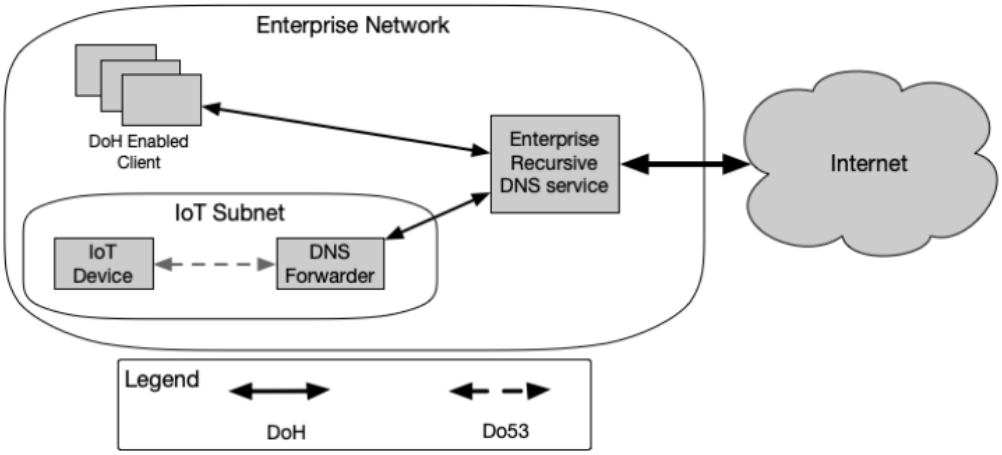

Fig. 2. Mixed use of Legacy DNS over UDP port 53 (Do53) and DoH

Figure. Mixed encrypted and legacy DNS in one enterprise. The IoT subnet cannot speak DoH directly, so its forwarder becomes both a trust boundary and a telemetry choke point; DoH-capable clients connect to the same controlled recursive service. Source: NIST SP 800-81 Rev. 3, Figure 2, public domain (U.S. government work).

1.2 Common Attack Vectors and Impact

The attack classes in this space are not hypothetical. Three incidents, spanning DNS registrar compromise, two flavors of Border Gateway Protocol (BGP) route hijacking, illustrate the pattern well enough to organize the rest of the handbook around it: an upstream naming or routing layer is compromised, a front end is silently substituted, and users sign transactions against a contract the attacker controls, with no bug in the audited smart contract at any point.

On April 24, 2018, an unknown actor exploited a BGP route leak at an upstream Internet Service Provider (ISP) called eNet, which announced a subset of Amazon Route 53's Internet Protocol (IP) address prefixes to its peers. Peered networks that had no reason to distrust the announcement began routing DNS queries for `myetherwallet.com` to a rogue DNS server, which answered with the address of a phishing clone hosted in Russia. The window lasted about two hours and netted the attacker 215 ETH, worth roughly $152,000 at the time (Internet Society, "What Happened? The Amazon Route 53 BGP Hijack"; CyberScoop). Amazon confirmed Route 53 itself was never breached: the compromise happened entirely at the routing layer, upstream of any control MyEtherWallet could apply to its own domain.

On February 3, 2022, KLAYswap users lost approximately $1.9 million in a two-hour window when attackers hijacked the autonomous system announcing routes to `developers.kakao.com` (AS9457) and served a modified `kakao.min.js` software development kit (SDK) file to KLAYswap's front end specifically, filtering by referrer so other consumers of the same SDK were untouched. The injected code waited for a user to initiate a swap, deposit, or withdrawal, then redirected the asset to an attacker wallet, laundering proceeds through OrbitBridge and FixedFloat (The Record; CertiK). This is the same BGP hijack primitive as MyEtherWallet, aimed one layer deeper, at a third-party JavaScript dependency instead of the primary domain.

On August 9, 2022, Curve Finance's front end was compromised through its DNS provider, `iwant-myname.com`: the attacker altered nameserver records for `curve.fi`, pointed the domain at a cloned interface, and had it request a malicious token approval from every visitor who connected a wallet during the window. Eight victims lost a combined $570,000-plus in USDC and DAI, later laundered through FixedFloat and Tornado Cash (CryptoTimes; Decrypt). Curve's own audited contracts were never touched; the registrar account was the entire attack surface.

Vector	Mechanism	Where it sits	Representative incident
Registrar or registry compromise	Attacker gains control of the domain account or nameserver delegation and repoints it	DNS, registrar layer	Curve Finance, Aug 2022
Route hijack (BGP) against the origin	Attacker announces false routes to hijack traffic destined for an authoritative DNS or hosting IP	Internet routing, upstream of DNS	MyEtherWallet, Apr 2018

Vector	Mechanism	Where it sits	Representative incident
Route hijack (BGP) against a dependency	Same primitive, aimed at a third-party script host the front end trusts	Dependency delivery	KLAYswap, Feb 2022
Cache poisoning	Off-path attacker injects a forged record into a resolver's cache, no registrar access needed	DNS resolution	Structural risk; see Part 2
ENS resolver or subdomain takeover	Attacker gains control of a resolver contract, a wildcard subdomain, or an expired-and-reclaimed name	ENS layer	See Part 3 threat model
IPFS gateway or pinning compromise	Gateway serves stale or substituted content, or a pin lapses and content becomes unavailable	IPFS layer	See Part 4 threat model
Domain or registration lapse	Domain expires or auto-renewal fails and a third party re-registers it	Registrar layer	See Part 2, Playbook 5

Impact is consistently the same shape regardless of which layer failed: the smart contract logic is untouched, the audit trail on-chain is technically correct, and the loss occurs entirely because the thing the user trusted to deliver correct instructions delivered attacker instructions instead. OWASP's Web3 Attack Vectors Top 15 names this category directly as **WA13, DNS, Domain, and Routing Infrastructure Hijacking**, and ranks it alongside supply chain compromise (WA02) and UI/UX approval phishing (WA06) as a non-contract risk that recurs across the industry precisely because it bypasses every control a contract audit can offer.

1.3 Relationship to SCSVS and Infrastructure Handbook

This handbook sits deliberately outside the Smart Contract Security Verification Standard (SCSVS), the Smart Contract Security Testing Guide (SCSTG), and the Smart Contract Weakness Enumeration (SCWE), because none of those catalogs address DNS resolution, registrar custody, ENS resolver correctness, or IPFS gateway trust. SCSVS structures its requirements across eleven domains, among them SCSVS-ARCH (architecture, design, and threat modeling) and SCSVS-COMM (secure interactions and communications), and both domains implicitly assume that the channel delivering the front end to the user, and the name the user resolved to reach it, are already trustworthy. That assumption is exactly the gap this handbook closes: a system can satisfy every SCSVS-COMM control on message integrity between the front end and the contract and still lose every user's funds the moment the front end itself is served from a hijacked domain. The playbooks in Parts 2 through 5 give an incident responder the DNS-, ENS-, and IPFS-specific procedures that SCSVS treats as a precondition rather than a verification target.

The closest sibling in the series is the **Infrastructure Security Handbook (07)**, which covers the servers, cloud accounts, secrets management, and network controls that DNS records and hosting configuration ultimately point to. Where this handbook asks "does the name resolve to the right place," Handbook 07 asks "is the place itself secured." A registrar account takeover (this handbook) and a compromised cloud Identity and Access Management (IAM) role (Handbook 07) are different root causes that can produce the identical user-facing outcome, a substituted front end, so a mature security program treats them as a matched pair of controls rather than either one alone. This handbook also overlaps at its edge with the **CDN and Front-End Supply Chain Security Handbook (01)**: once DNS correctly resolves to your CDN, Subresource Integrity, Content Security Policy, and build-pipeline provenance take over as the controls that keep the served content trustworthy, a boundary made explicit in that handbook's own Part 1. Finally, every playbook here is a specialization of the general process defined in the **Incident Response Handbook (06)**: this book supplies the domain-, resolver-, and gateway-specific detection and containment steps, while Handbook 06 supplies the surrounding communication, evidence-preservation, and post-incident review process that applies regardless of which system failed.

1.4 How to Use This Handbook

Treat this Part as the only chapter meant to be read start to finish; everything after it is reference material organized to be entered at any point during normal operations or during an active incident. Part 2 covers DNS itself: its threat model, DNSSEC hardening with the DS, RRSIG, and DNSKEY chain of trust, Certification Authority Authorization (CAA) records, and three playbooks for a suspected hijack, a lapsed domain, and DNS-hosted abuse of your brand. Part 3 does the same for ENS: data integrity and ownership, the gasless DNSSEC-via-CCIP-Read bridge back to Part 2's chain of trust, subdomain and resolver takeover, and a recovery playbook. Part 4 covers IPFS: content addressing as an integrity guarantee, the separate and weaker guarantee of gateway and pinning availability, and a playbook for a gateway or pinning incident. Part 5 folds in conventional hosting, the overlap with Handbook 01's CDN controls, a defacement playbook, and the handoff into Handbook 06's incident response process.

If you are hardening a system before launch, read Parts 2 through 5 in order and work each chapter's checklist into your deployment runbook. If you are responding to a live incident, skip directly to the matching playbook, Part 2's Chapter 4 for a suspected DNS hijack, Part 2's Chapter 5 for a lapsed or stolen domain, Part 3's Chapter 9 for a compromised ENS name, or Part 4's Chapter 13 for an IPFS gateway or pinning failure, and use Part 6's Appendix C as a one-page router if you are not yet sure which system failed. Appendix A defines every acronym and term used across all five Parts in one place, and Appendix B consolidates every hardening checklist in this handbook into a single pre-launch and periodic-review document.

Key controls for Part 1

- Treat DNS, ENS, and IPFS resolution as security-critical, not merely operational: map exactly which naming path each user-facing surface takes before deciding which Part's controls apply.
- Recognize the shared attack pattern behind MyEtherWallet (2018), KLAYswap (2022), and Curve Finance (2022): the contract layer stayed intact in all three; the naming or routing layer in front of it did not.
- Cross-reference SCSVS-ARCH and SCSVS-COMM assumptions against this handbook's playbooks rather than treating contract verification as sufficient on its own.
- Pair this handbook with Infrastructure Security (07) for the systems DNS records point to, with CDN and Front-End Supply Chain Security (01) for what happens after DNS resolves correctly, and with Incident Response (06) for the response process each playbook here specializes.

Part II: DNS Security

The Domain Name System (DNS) translates the name a user types or clicks into the IP address their browser connects to, and every control described elsewhere in this handbook sits downstream of that translation. A team can pin every script with Subresource Integrity and audit every contract line by line, and still lose its user base the moment an attacker moves the domain, because the browser never reaches the point of checking a pinned hash against the real origin. This part builds the DNS threat model for Web3 teams, covers the hardening controls that close the gap (DNS Security Extensions (DNSSEC), Certification Authority Authorization (CAA), registrar lock, and monitoring), and closes with three field playbooks: hijack, registrar or expiry loss, and brand-abuse phishing.

2. DNS Threat Model

DNS was designed in the 1980s for availability and simplicity, not for authentication, and every control in this part exists to compensate for that original design choice. This chapter separates the ways DNS itself can be subverted from the ways the surrounding registration ecosystem can be subverted, because the two failure classes demand different defenses.

2.1 Hijacking, Poisoning, Abuse

Three distinct failure modes get grouped under "DNS attacks" and each needs its own defense. **DNS hijacking** is unauthorized modification of a domain's authoritative records: an attacker who gains write access to a zone (via a compromised registrar account, DNS provider, or hijacked route to the provider's infrastructure) changes the Name Server (NS), A, or AAAA records so queries resolve to attacker-controlled infrastructure. It is the highest-impact variant, since it redirects every query, from every resolver, everywhere, until fixed. **DNS cache poisoning** (spoofing) is narrower: an on-path or off-path attacker injects a forged response into a resolver's cache without touching the authoritative zone, exploiting weak randomization of the query ID and source port a resolver uses to match responses to requests. Researcher Dan Kaminsky's 2008 disclosure showed this could be executed far faster than assumed, forcing an emergency patch cycle around source port randomization, the practical predecessor to DNSSEC's fix (Wikipedia,

DNS spoofing). **DNS abuse** is the broadest category and the one ICANN uses contractually: malware, botnets, phishing, pharming, and spam as a delivery mechanism for the other four (ICANN SAC115). It needs no compromise of your own DNS; a typosquat or homograph domain copying your brand is abuse against your users even though your zone was never touched (Chapter 6).

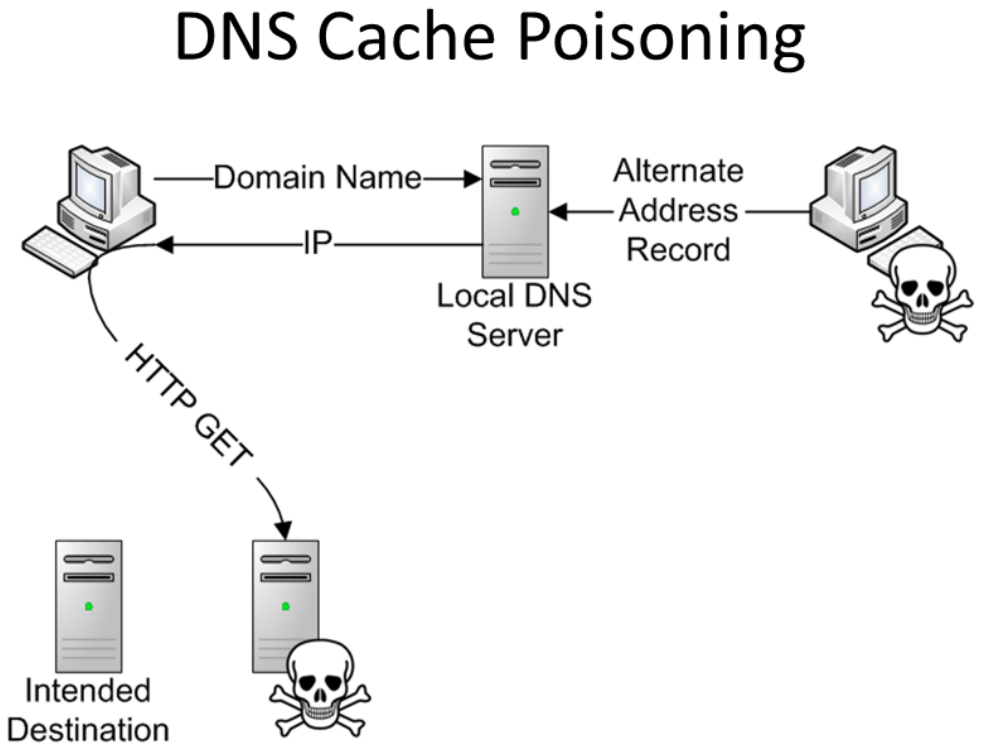

Figure. Cache poisoning does not require any access to a registrar or authoritative zone; it only requires the attacker's forged answer to reach the resolver's cache before the legitimate one does. Source: Wikimedia Commons, CC BY-SA 4.0.

The April 2018 attack on MyEtherWallet illustrates hijacking at the routing layer rather than the registrar layer. An internet service provider identified as eNet, based in Columbus, Ohio (or a customer operating through its network), announced false Border Gateway Protocol (BGP) routes claiming roughly 1,300 IP addresses across five blocks belonging to Amazon Web Services, including addresses dedicated to Amazon's Route 53 DNS service. Peering partners, including Hurricane Electric, propagated the false routes, so queries that should have reached Amazon's real name servers were answered by an attacker-controlled server. Between 11:05 AM and 1:03 PM UTC on April 24, 2018, victims were served a phishing clone; approximately $150,000 in Ethereum was stolen before the hijack was withdrawn (The Register; Wikipedia, BGP hijacking). No registrar or application code was compromised: the attack lived entirely in internet routing, one layer below every control this handbook otherwise discusses.

Vector	Level	Typical mechanism	Case
Registrar account takeover	Registrar	Phishing, credential stuffing, SIM-swapped 2FA	Handbook 05 (insider/DPRK)
DNS provider compromise	Provider	Stolen API keys, compromised nameserver infra	Curve Finance, 2022 (2.2)
BGP route hijack	Network routing	False route announcements propagated by peers	MyEtherWallet, 2018 (above)
Cache poisoning	Resolver	Forged response matching guessed query ID/port	Kaminsky-class, pre-DNSSEC
Registry compromise	Registry (ccTLD/gTLD)	State-actor intrusion into registry systems	Sea Turtle (2.2)
Brand-impersonation abuse	Attacker's own DNS	Typosquat, IDN homograph domains	Chapter 6

Figure 1. The DNS hijack chain from credential or routing compromise to on-chain loss. DNSSEC validation is the one checkpoint on this path that can independently reject the forged answer.

2.2 Registrar and Registry Risks

The domain name system has an administrative hierarchy that mirrors its technical one. ICANN accredits **registries** (operators of a top-level domain, such as Verisign for `.com` and `.net`) and **registrars** (retail-facing companies, such as GoDaddy or Namecheap, that sell domains and submit changes to the registry over the Extensible Provisioning Protocol, EPP). A Web3 team's domain security depends on the weakest link in that chain, and the two levels fail differently.

Registrar-level compromise is the far more common incident. An attacker who phishes, credential-stuffs, or social-engineers their way into the account that manages a domain (or a separate DNS provider account, if the team delegates nameservers away from the registrar) can rewrite records directly, no cryptographic bypass required. Curve Finance's August 9-10, 2022 incident is the canonical Web3 example: the team's DNS provider, iwantmyname, had its nameservers compromised (CEO Michael Egorov described it as "ns compromised," a nameserver-level rather than account-level breach). Attackers redirected `curve.fi` to a cloned front end that prompted visitors to approve a malicious contract; roughly $575,000 (340 ETH) was drained before the team redirected users to its uncompromised alternate interface, `curve.exchange`, and coordinated on revoking approvals (rekt.news, Curve Finance). The postmortem line that circulated afterward, "Web3 still runs on Web2," is the thesis of this handbook.

Registry-level compromise is rarer but far more dangerous, since it can affect every domain under that registry rather than one registrant's account. The Sea Turtle campaign, documented by Cisco Talos in 2019, was the first publicly known case of a state-sponsored actor compromising a domain registry itself for espionage: active from January 2017 through at least early 2019, the actor compromised registrars, registries, telecoms, and ISPs across at least 40 organizations in 13 countries, primarily to reach government and energy-sector targets in the Middle East and North Africa. Targets included a ccTLD registry and Netnod's Swedish infrastructure, which handles information related to root server operations; the actor then used fraudulent Let's Encrypt and Comodo certificates to intercept victims after redirecting their DNS (Cisco Talos, Sea Turtle). MITRE ATT&CK catalogs this pattern as T1584.002, Compromise Infrastructure: DNS Server (MITRE ATT&CK T1584.002). Registry compromise is out of a Web3 team's direct control; the defense is DNSSEC (Section 3.1), which makes a forged answer detectable regardless of layer, plus registry lock (3.3) for domains warranting the strongest protection. Credential hygiene for registrar-access holders (Handbook 03) is the first line of defense against the far more common registrar-level scenario.

Mitigation scales with the risk level: hardware-key MFA and client-side registrar lock for the account tier, provider vetting and NS-drift monitoring for the DNS-provider tier, and registry lock plus DNSSEC chain validation for the registry tier, since that top tier is the one layer no registrant-side credential hygiene can reach.

---

3. Hardening and Prevention

Each hardening control in this chapter closes a specific gap identified in Chapter 2. None of them is sufficient alone; together they form layered defense against a threat model where the domain itself is the target.

3.1 DNSSEC: Chain of Trust, DS, RRSIG, DNSKEY

DNSSEC adds origin authentication and data integrity to DNS responses through digital signatures; it adds no confidentiality, so signed queries and responses remain visible on the wire. The core specification spans RFC 4033 (introduction and requirements), RFC 4034 (resource records), and RFC 4035 (protocol modifications), later clarified by RFC 6840 and consolidated as an internet Best Current Practice in RFC 9364 (BCP 237). Four record types do the work. A **DNSKEY** record publishes a zone's public signing keys, conventionally split into a Zone Signing Key for day-to-day records and a Key Signing Key that signs the DNSKEY set itself. An **RRSIG** record is the signature over a specific resource record set (all the A records for a name, for instance), generated with the zone's private key. A **DS** (Delegation Signer) record, published in the *parent*

zone, holds a cryptographic hash of the *child* zone's Key Signing Key, linking a domain's signature to a chain of trust a resolver can walk to the root. The root zone has been signed since 2010, giving every validating resolver a built-in trust anchor to start from.

Figure 2. The DNSSEC chain of trust. Each DS record in a parent zone commits to the hash of the child zone's signing key, so a validating resolver can trace trust from the root anchor to any signed record.

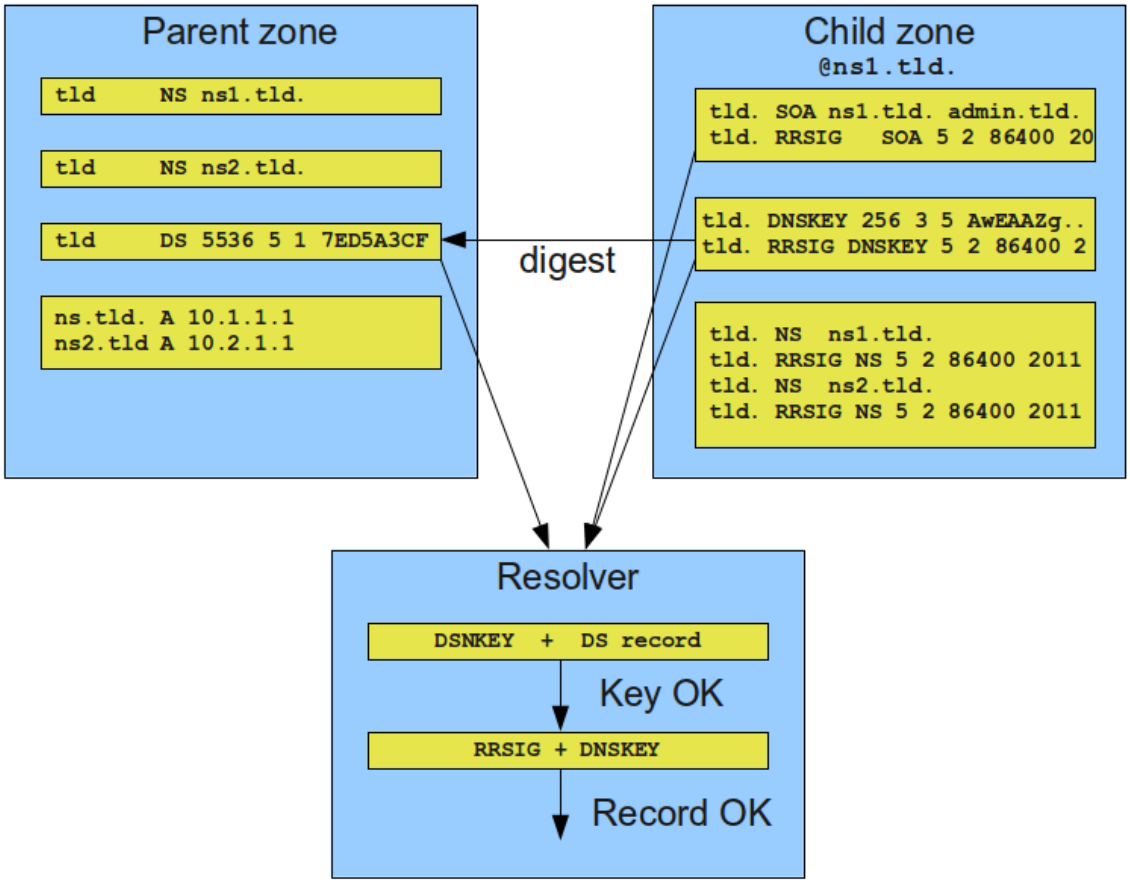

Figure. A concrete walkthrough of the same chain from Figure 2, with actual record types: the resolver digests the child zone's DNSKEY, matches it against the parent's DS record, then uses the validated DNSKEY to check the RRSIG over the answer itself. Source: Wikimedia Commons, CC0 1.0 (public domain).

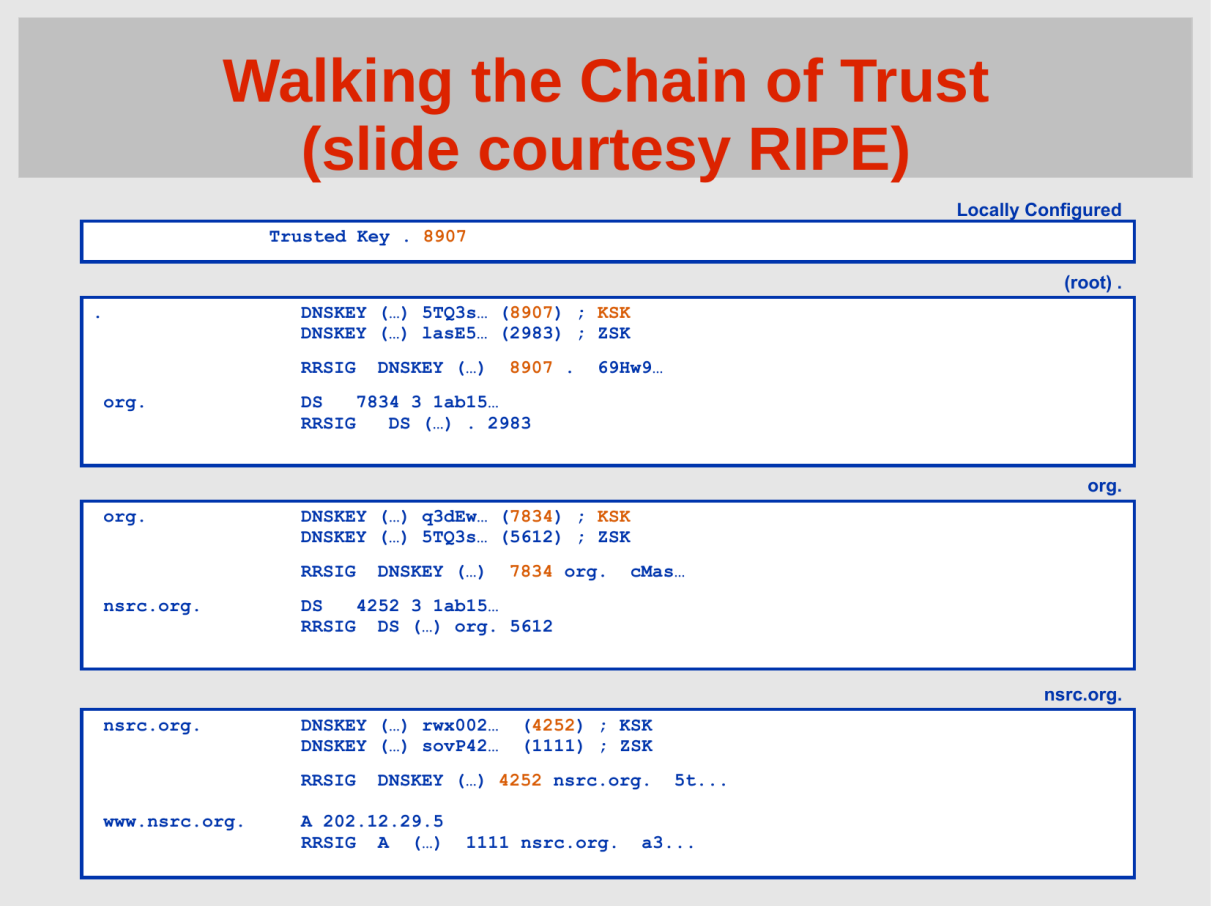

Figure. A record-level DNSSEC chain walk. The resolver begins with a locally configured root key, verifies the root DNSKEY set, follows the parent's DS digest into the child DNSKEY set, and finally validates the RRSIG over the requested address record. This is the concrete evidence behind an `AD` *result, not merely a diagram of zone hierarchy. Source: ICANN DNSSEC Training, Walking the Chain of Trust, slide credited to RIPE; reproduced with attribution for technical commentary.*

A resolver that validates DNSSEC will query and check this chain automatically; a client can also inspect it directly:

```
# Query with DNSSEC records requested and check for the RRSIG
dig +dnssec example.com A

# Use delv, the DNSSEC-aware companion to dig, for an explicit validation verdict
delv example.com A
# ;; fully validated
```

An operator signs a zone with tooling built into most authoritative DNS software (BIND's `dnssec-signzone`, or a managed "enable DNSSEC" toggle most modern providers offer), then submits the resulting DS record to the registrar for publication in the parent zone. That hand-off is the one manual step that breaks most deployments: a zone signed locally but with no DS record published upstream is validated by nobody.

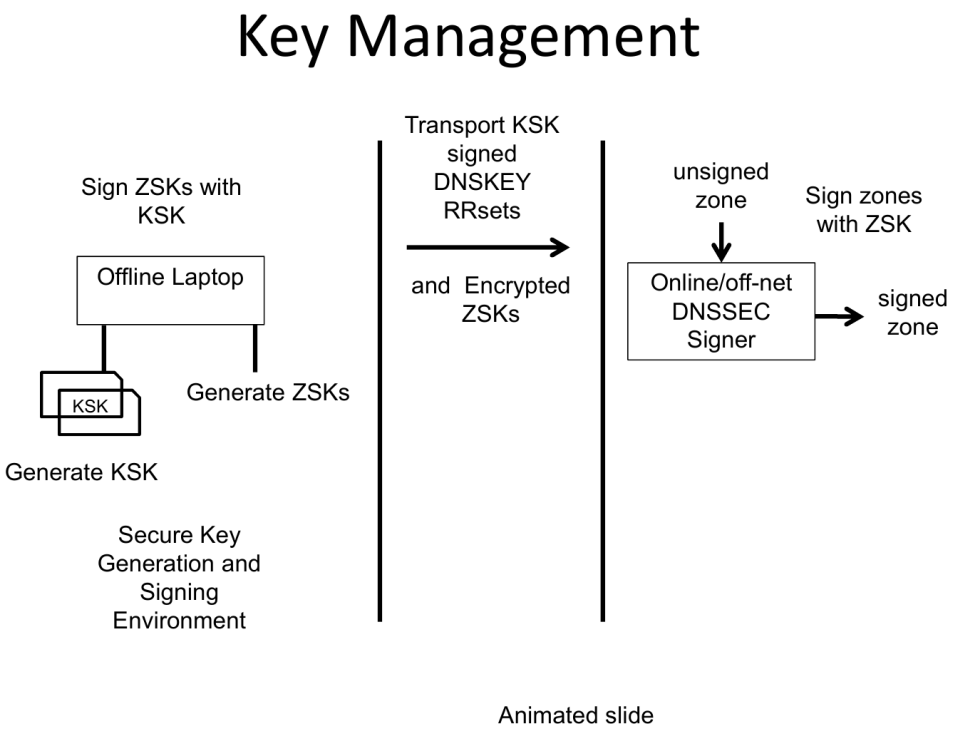

Figure. A practical split-key DNSSEC signer design. The Key Signing Key remains offline and signs only DNSKEY RRsets; encrypted Zone Signing Keys and an unsigned zone cross into an isolated signer that emits the signed zone. Compromise of the online signer therefore does not automatically expose the long-lived trust anchor. Source: ICANN DNSSEC Design and Operations Training, Key Management; reproduced with attribution for technical commentary.

3.2 Why DNSSEC Matters for Web3 (Wallet Redirection, Phishing, MITM)

For most web applications, a DNS-layer redirect to a phishing clone costs a user their password, which they can reset. For a Web3 front end, the same redirect costs a user their private key material or a signed, irreversible transaction, because the front end's entire job is constructing a transaction and asking a wallet to sign it. **Wallet redirection** happens when a hijacked or poisoned resolution serves JavaScript that alters the transaction the wallet is asked to sign, exactly as in the Curve Finance incident, where the malicious clone prompted an `approve()` call to an attacker-controlled contract. **Phishing** through DNS is more dangerous than a typosquat, because the browser's address bar shows the real, trusted domain; there is no lookalike spelling for a user to catch. **Machine-in-the-middle (MITM)** describes an on-path attacker, whether a rogue resolver reached via BGP hijack (the MyEtherWallet mechanism) or one pushed by a compromised public Wi-Fi network, silently substituting answers for every query that crosses it.

DNSSEC directly defeats the MyEtherWallet-style attack: the rogue server the BGP hijack routed traffic to had no way to produce validly signed responses for the real zone, so a validating resolver checking the chain in Figure 2 would have rejected the forged answers with SERVFAIL instead of serving them. Be precise about what DNSSEC does *not* cover: in the Curve Finance

case, the attacker compromised the DNS provider's own nameserver infrastructure and could potentially re-sign records with keys the provider itself controlled. DNSSEC authenticates that a response came from whoever holds the zone's private keys; it says nothing about whether that key holder was itself compromised. This is why it is necessary but not sufficient, and why Section 3.3's registrar and registry lock controls, which govern *who can change the DS record and NS delegation in the first place*, matter as much as signing itself. SRI (Handbook 01) protects a page's own script integrity once the correct origin is reached; DNSSEC is the layer that gets the browser to the correct origin at all.

3.3 CAA, Registrar Lock, and Monitoring

Three controls close the administrative half of the threat model: who can issue a certificate for your domain, who can change your DNS delegation, and how quickly you find out if either happens without authorization.

CAA (Certification Authority Authorization), defined in RFC 8659, is a DNS record naming which Certificate Authorities may issue TLS certificates for a domain. Since the CA/Browser Forum's Ballot 187 passed in March 2017 and took effect on September 8, 2017, checking CAA records before issuance has been mandatory for every publicly trusted CA under the Baseline Requirements; a CA that ignores a restrictive record and issues anyway violates its own accreditation. This closes the gap where a compromised or careless CA issues a certificate for your domain to someone else.

```
; Only the named CA may issue standard certificates
example.com. CAA 0 issue "letsencrypt.org"
; No CA may issue wildcard certificates at all
example.com. CAA 0 issuewild ";"
; Send violation reports here
example.com. CAA 0 iodef "mailto:security@example.com"
```

Registrar and registry lock governs the delegation itself. At the registrar level, EPP status codes such as `clientTransferProhibited`, `clientUpdateProhibited`, and `clientDeleteProhibited` are set on the registrant's behalf and block transfers, updates, or deletion until explicitly lifted, usually requiring an out-of-band step beyond the portal password. **Registry lock** is the stronger version: `serverTransferProhibited`, `serverUpdateProhibited`, and `serverDeleteProhibited` are set at the registry itself, so even a fully compromised registrar account cannot push a change through without separate, typically multi-party, authorization directly with the registry operator. It is not available for every TLD or registrar tier, but for a domain serving a production Web3 front end it is worth the overhead.

Monitoring turns both controls into detection rather than pure prevention. Certificate Transparency (CT) logs, queryable through services like crt.sh, record every publicly trusted certificate issued; a live CT stream (via tools such as Certstream) surfaces a certificate for your domain, or a lookalike domain, within minutes of issuance, catching a CAA violation and a

typosquat campaign in the same feed. Pair this with scheduled WHOIS/RDAP lookups for registrant or nameserver changes and a `dig` diff against a known-good baseline for record drift outside a change window.

3.4 Deployment Reality: Low DNSSEC Adoption and Resolver Validation

DNSSEC has been standardized for two decades and the root has been signed since 2010, yet deployment remains genuinely low: RFC 9364 itself states that fewer than 10% of websites use DNSSEC despite the protocol's status as internet Best Current Practice. The picture varies sharply by TLD. Roughly 45% of country-code TLDs support it, and a handful, `.nl`, `.cz`, `.no`, `.se`, and `.nu`, exceed 50% adoption among second-level domains, largely because their registries built pricing incentives or default-on policies into the registrar channel. The generic TLDs most Web3 projects actually register under tell a different story: only about 4% of `.com` domains and 5% of `.net` domains are signed (Wikipedia, DNSSEC).

Signing your own zone is only half the equation, because a signature nobody checks provides no protection. APNIC Labs, measuring validation behavior from the resolver side, puts the global DNSSEC validation rate at roughly 30% of users, and estimates about half of that traffic comes from just two public resolvers, Google's `8.8.8.8` and Cloudflare's `1.1.1.1`, validating on behalf of users who configured nothing themselves (APNIC blog). Validation clusters unevenly by country, high in parts of Africa and Scandinavia, low in infrastructure-heavy regions including the UK, Canada, Spain, and China. The practical consequence: signing your zone protects roughly a third of your users, and which third depends on whose resolver they happen to use that day, not on anything the team controls. Treat DNSSEC as one layer, not a complete answer, and prioritize registrar/registry lock and monitoring as controls that protect all users regardless of resolver.

3.5 Monitoring and Anomaly Detection

Detection latency is the variable that separates a contained incident from a mass-casualty one; both MyEtherWallet and Curve Finance ran for hours before being caught and mitigated by outside researchers and the community. Build alerting around a small set of signals that catch the failure modes in Chapter 2 before users report them.

Signal	Detection method	Response trigger
NS/A/AAAA record change	Scheduled `dig` diff vs. known-good baseline, or DNS monitoring SaaS	Any change outside a declared maintenance window
Unauthorized certificate issuance	CT log stream (crt.sh, Certstream) filtered on domain and near-miss lookalikes	Any cert from a CA not in your CAA `issue` list
WHOIS/RDAP change	Scheduled RDAP lookup	Registrant, nameserver, or EPP status change you did not initiate
DNSSEC validation failure	`delv` or resolver-side validation monitoring	SERVFAIL on your own signed records

Signal	Detection method	Response trigger
TTL anomaly	Compare served TTL against baseline	Sudden drop, often used to enable fast record flips
Lookalike domain registration	Typosquat scan (`dnstwist`) against brand terms	New registration within edit distance 1-2 of brand

Route the highest-confidence signals, certificate issuance and NS/A record drift, into a paging alert, not a daily digest; the incident cost is measured in minutes, as the 2018 and 2022 cases both show.

---

4. Playbook: Suspected DNS Hijack or Poisoning

This playbook applies when your own domain's resolution has been altered, whether through registrar compromise, DNS provider compromise, or cache poisoning. It specializes the general process defined in the Incident Response Handbook (06) for this specific failure mode.

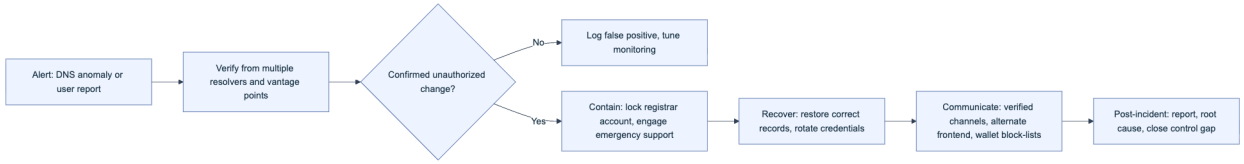

Figure 3. The DNS hijack response cycle. Verification from more than one vantage point is the gate before any containment action, since a single resolver's cached bad answer can look identical to a real hijack.

4.1 Detection and Verification

The first job is ruling out a false alarm before taking disruptive containment action, because rotating credentials and pushing emergency record changes has its own operational cost. Query the domain from several independent vantage points and resolvers rather than trusting a single lookup, since a single poisoned cache (Section 2.1) can produce a false positive that looks identical to a real hijack from one location.

```
# Compare NS answers across independent public resolvers
dig @8.8.8.8 example.com NS +short
dig @1.1.1.1 example.com NS +short
dig @9.9.9.9 example.com NS +short

# Confirm against the registry's authoritative delegation directly
dig @a.gtld-servers.net example.com NS +short

# Corroborate with a DNSSEC validation check and a CT log query
delv example.com A
curl -s "https://crt.sh/?q=example.com&output=json" | head
```

If independent resolvers and the authoritative registry agree with your known-good baseline, the anomaly was local caching, not a hijack. If they disagree, escalate immediately: this is not a wait-and-confirm-again situation, given how fast both reference incidents in this part caused loss.

4.2 Containment and Recovery

Move in parallel, not sequentially: every minute the malicious record is live is a minute of user exposure. Contact the registrar's emergency or security escalation channel (established in advance) to freeze further changes and confirm the legitimate account holder's identity out of band. Simultaneously rotate credentials: reset the registrar password, rotate any DNS provider API keys, and revoke active sessions, since the same compromised credential can be reused to undo your fix. Restore correct NS, A, DS, and CAA records from your documented baseline, and if the incident involved BGP-level manipulation, engage your network provider and upstream transit partners to withdraw the false route (Resource Public Key Infrastructure (RPKI) route origin validation, where deployed, limits how far a false announcement propagates). If the attacker obtained a valid TLS certificate during the incident (per the CT check in Section 4.1), request revocation from the issuing CA immediately rather than waiting for expiry. Confirm registrar and registry lock (Section 3.3) are re-enabled once control is regained, since the fix itself may have required disabling them.

4.3 Communication and Post-Incident

Users cannot distinguish a fixed domain from a still-compromised one without your help, so communicate over channels an attacker cannot spoof: a pre-established, previously-verified social media account, a Discord with role verification, and, if one exists, an alternate front end on genuinely independent infrastructure, exactly as Curve Finance directed users to `curve.exchange` while `curve.fi` was compromised. Publish any malicious contract address so users can check their approvals against it, and submit that address to wallet-side phishing feeds and scam-address lists so wallets warn users automatically. Advise revoking token approvals through a reputable tool rather than assuming disconnection is sufficient, since an `approve()` call outlives the session that created it. Close the loop with a written post-incident report covering root cause, timeline, and the specific control gap that allowed the incident, and feed that gap into the monitoring table in Section 3.5.

---

5. Playbook: Domain Expiry or Registrar Compromise

This playbook covers two distinct but related failure modes that both end with someone other than your team controlling the domain: an expired registration that lapses into someone else's hands, and a registrar account compromise severe enough that the attacker changes registrant contact information or initiates a transfer rather than merely editing DNS records (the narrower scenario in Chapter 4).

Expiry is the quieter of the two and the more preventable. A domain that is not renewed enters an Auto-Renew Grace Period and then a Redemption Grace Period under ICANN policy, giving the registrant a window (typically up to 30 and then 30 more days, varying by registry) to reclaim it before general release. Once released, automated "drop-catching" services often re-register high-value domains within seconds, and the original owner then has no special claim to it. The trigger is almost always mundane: an expired payment card, a registrant contact email silently bouncing, or a domain registered by a departed employee whose account access lapsed with their employment (the Employee Lifecycle Handbook, 03, covers this offboarding gap). Registrar compromise escalates the failure: an attacker with sufficient account access can change the registrant email, weaken or disable transfer locks, and initiate an outbound transfer to a different registrar, at which point recovery moves from an internal fix to a formal dispute.

Trigger	Immediate action	Escalation path	Recovery mechanism
Renewal payment failure	Update payment method same day	Registrar billing support	Renewal inside Auto-Renew Grace Period, no dispute needed
Registrant contact bouncing	Correct WHOIS/RDAP contact immediately	Registrar account support	Prevents loss of renewal/recovery notices
Domain expired and released	Attempt drop-catch or direct re-registration	Registrar, then a drop-catch service	Redemption Grace Period reclaim, or repurchase
Account compromised, records altered only	Emergency support, credential rotation	Registrar security/abuse team	Records restored from audit log; see Chapter 4
Account compromised, transfer initiated	Contact both losing and gaining registrar	ICANN Transfer Dispute Resolution	Reversal within the dispute window if filed promptly

Prevention outruns recovery in every row of that table. Set registrations to multi-year terms with auto-renewal on a payment method that is actively monitored, keep WHOIS/RDAP contacts current and separately monitored (not tied to a single departing employee's inbox), and enable both registrar-level and, where available, registry-level lock as a standing default rather than something enabled only after a scare. Treat domain renewal and access review as a recurring calendar item owned by a named individual, not an ad hoc task, since the failure mode here is neglect rather than a sophisticated attack.

6. Playbook: DNS Abuse (Phishing, Malware Hosting on Your Brand)

This playbook differs from the two before it in a specific way: your own DNS was never touched. Instead, an attacker registers infrastructure that impersonates your brand, whether through a typosquat domain (`examp1e.com`, `example-io.com`), an Internationalized Domain Name (IDN) homograph that renders visually identical to your real domain using lookalike characters from another script, or a phishing kit hosted on entirely unrelated infrastructure that simply copies your visual identity. The user's DNS resolution worked correctly; they just resolved the wrong name.

Detection here is proactive rather than reactive, because there is no anomaly on your own infrastructure to alert on. Run scheduled typosquat sweeps against your brand terms and common misspellings using a tool such as `dnstwist`, which generates permutations (character insertion, deletion, transposition, and IDN homograph variants) and checks which ones are registered:

```
# Generate and check likely typosquat/homograph variants of a brand domain
dnstwist --registered example.com

# Cross-reference against Certificate Transparency for lookalikes that
# have gone as far as obtaining a TLS certificate
curl -s "https://crt.sh/?q=%25example%25&output=json" | head
```

Once a malicious domain is confirmed, response is a takedown and communication exercise rather than a DNS-recovery one. File an abuse report with the domain's registrar, whose abuse contact is a required, publicly listed field under ICANN accreditation terms, and separately with the hosting provider if the phishing content sits on different infrastructure than the domain's nameservers. Submit the URL to browser-level and wallet-level protection feeds (Google Safe Browsing, and Web3-specific phishing detection services that wallets such as MetaMask query before a connection or signature) so users are warned before reaching the fake page, independent of registrar or host response time. For domains that clearly infringe your trademark, the Uniform Domain-Name Dispute-Resolution Policy (UDRP), ICANN's standard dispute mechanism, provides a path to suspension or transfer, though it runs weeks rather than the hours a live phishing campaign needs contained. A short internal checklist keeps the response consistent regardless of who is on call:

- ☐ Confirm the domain or content is genuinely impersonating your brand, not a legitimate mention or parody
- ☐ Screenshot and archive the phishing page and its DNS/WHOIS record before it disappears
- ☐ File abuse reports with the domain registrar and, separately, the content host
- ☐ Submit the URL to Google Safe Browsing and relevant Web3 wallet phishing-detection feeds

- [ ] Post a verified-channel warning if the domain has begun circulating in your community
- [ ] Open a UDRP filing if the domain persists and clearly infringes your trademark
- [ ] Log the domain and pattern for the next `dnstwist` baseline update

---

Key controls for Part 2

- Sign your zone with DNSSEC and confirm the DS record is published at the registrar, then verify with `delv`, understanding it protects roughly a third of users depending on their resolver.
- Publish a restrictive CAA record naming only the CAs you actually use, and monitor Certificate Transparency logs for any issuance outside it.
- Enable registrar-level lock as a default; add registry lock for any domain serving production front-end traffic.
- Monitor NS/A record drift, WHOIS/RDAP changes, and typosquat registrations on a schedule, not only after an incident.
- Maintain a pre-verified, attacker-resistant communication channel and, where feasible, an independently hosted alternate front end for the moment your primary domain is compromised.

Part III: ENS (Ethereum Name Service)

The Ethereum Name Service (ENS) turns a 42-character hexadecimal address into a name a human can read and a wallet can display, but the system that makes `vitalik.eth` trustworthy is itself a set of smart contracts, an off-chain resolution path, and a permission model that can be misconfigured or hijacked in ways that echo classic DNS failures with different mechanics. This part treats ENS as the Web3-native complement to the DNS material in Part II: the same integrity, ownership, and resolver-trust questions apply, but the failure primitives are resolver takeover instead of registrar hijack, and fuse mismanagement instead of TTL abuse. It works through ENS's data model, its bridge back into traditional DNS, the cross-chain resolution it now supports, the concrete ways a name or resolver gets compromised, a response playbook, and an integration checklist for teams that resolve ENS names in a front end or backend service.

7. ENS Security Overview

ENS is a set of Ethereum smart contracts, standardized starting with EIP-137, that map human-readable names to resources: addresses, content hashes, and arbitrary text records. It is not a fork of DNS; it is a purpose-built registry with its own ownership, expiry, and resolution rules that happen to interoperate with DNS at specific integration points.

7.1 Data Integrity and Ownership

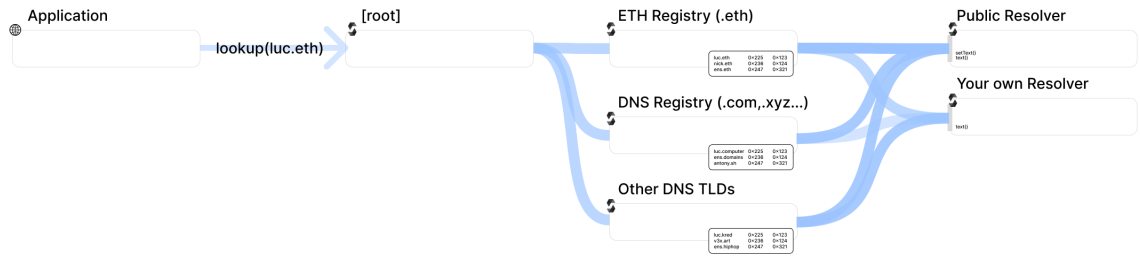

Figure. ENS resolution is a two-contract decision, not a single lookup: the client namehashes the name, asks the Registry which resolver is authoritative for that node, and then asks that resolver for the requested record. Monitoring only Registry ownership misses a resolver substitution, while monitoring only address records misses a resolver change. Source: ENS Documentation, Resolution, documentation visual used with attribution.

Every ENS name reduces to a fixed-length identifier called a node, computed with the `namehash` algorithm defined in EIP-137: hash the empty string to get the root, then recursively hash each label from right to left and combine it with the accumulated hash of its parent. `vitalik.eth` and `mysite.example.eth` both resolve to nodes in constant time regardless of how many labels they contain, which is what lets a single Registry contract index the entire namespace (EIP-137).

The ENS Registry is the source of truth for exactly three things per node: the owner, the resolver address, and a time-to-live (TTL) hint for caching. Nothing else lives in the Registry; addresses, content hashes, and avatars live in whatever resolver contract the node points to. Ownership itself is a bearer asset. Unwrapped second-level `.eth` names are ERC-721 tokens on the base registrar, and names wrapped with the NameWrapper (Section 7.4) are ERC-1155 tokens; either way, whoever holds the token controls the name, with no customer-support desk to call if the key is lost or stolen (ENS Docs: Architecture). `.eth` names must also be renewed on-chain before expiry or they enter a grace period and then return to the open market, an ownership-integrity failure mode that mirrors the domain-expiry risk covered in Part II's registrar playbook.

Registry field	Meaning	Who can change it
`owner(node)`	Controls the node: can set a new resolver, TTL, or transfer/reassign subnodes	Current owner or an approved operator
`resolver(node)`	Address of the contract that answers `addr()`, `text()`, `contenthash()` for this node	Current owner or an approved operator
`ttl(node)`	Suggested cache lifetime for resolved records	Current owner or an approved operator

```
# Read-only integrity check: confirm registry state for a name you control.
# NODE is the namehash of the name (EIP-137); ENS_REGISTRY is the current
# mainnet Registry address from the ENS deployments page - verify before use.
cast call "$ENS_REGISTRY" "owner(bytes32)(address)" "$NODE" --rpc-url "$RPC_URL"
cast call "$ENS_REGISTRY" "resolver(bytes32)(address)" "$NODE" --rpc-url "$RPC_URL"
```

7.2 ENS and DNSSEC: Gasless DNSSEC via CCIP Read

ENS bridges into traditional DNS so that anyone who already owns a DNS domain can claim the matching ENS name without a separate on-chain auction. Domain Name System Security Extensions (DNSSEC), the cryptographic chain of trust from the DNS root down to individual records covered in Part II, is what makes that claim verifiable rather than merely asserted.

The original path is expensive: a domain owner publishes a `TXT` record such as `_ens.example.com "a=0xYourAddress"`, then submits a DNSSEC proof on-chain to the `DNSRegistrar` contract, which a `DNSSECOracle` contract verifies signature by signature up the chain, at a gas cost that can run into the millions for a deep proof (ENS Docs: DNS Registry). ENSIP-17 (Gasless DNS Resolution) removes that cost by combining wildcard resolution with off-chain fetching. The domain owner instead publishes `TXT example.com "ENS1 <resolver-address>"` at the zone apex; when a client resolves `example.com` through ENS and finds no on-chain record, it invokes EIP-3668 (CCIP Read), fetching the DNSSEC proof from an off-chain gateway and passing it to a callback that verifies the signature chain on-chain before trusting it. The acronym CCIP here refers to the original 2020 “CCIP Read” offchain-lookup proposal in EIP-3668, not the unrelated Chainlink Cross-Chain Interoperability Protocol that shares the same initials; conflating the two is a common and avoidable confusion in Web3 documentation.

The security property that survives the shortcut is the one that matters: the gateway only supplies data, it never supplies trust. The callback still runs the DNSSEC verification on-chain, so a malicious or compromised gateway can withhold or delay a response but cannot forge a valid proof for a domain it doesn’t control (EIP-3668, Security Considerations). That distinction, gateway-as-transport versus gateway-as-authority, is the axis every other CCIP-Read resolver should be judged against, and it recurs in Section 7.4.

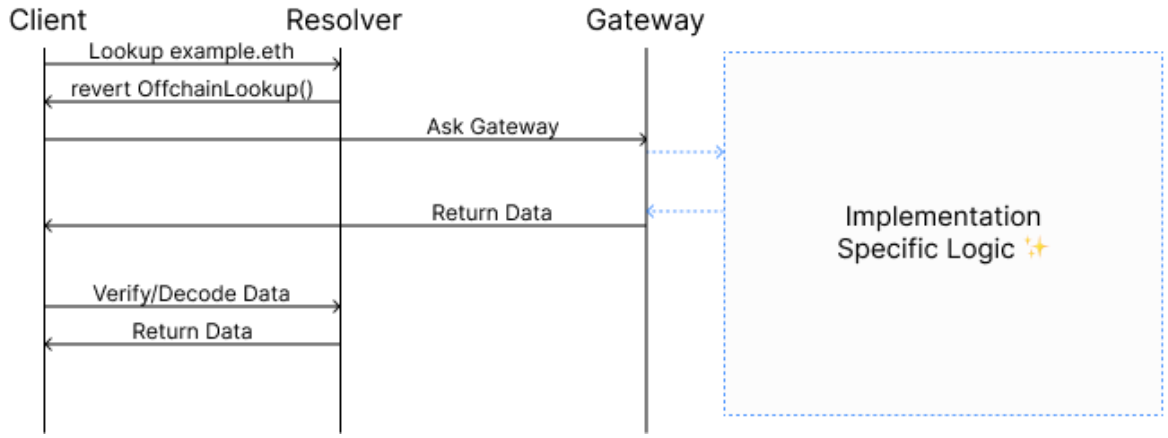

Figure. The official CCIP-Read sequence exposes the security boundary: the resolver returns `OffchainLookup`*, the client fetches untrusted data from the named gateway, and the callback validates that response before resolution completes. A client that skips or misroutes the callback converts a verifiable transport into direct gateway trust. Source: ENS Documentation, CCIP Read, documentation visual used with attribution.*

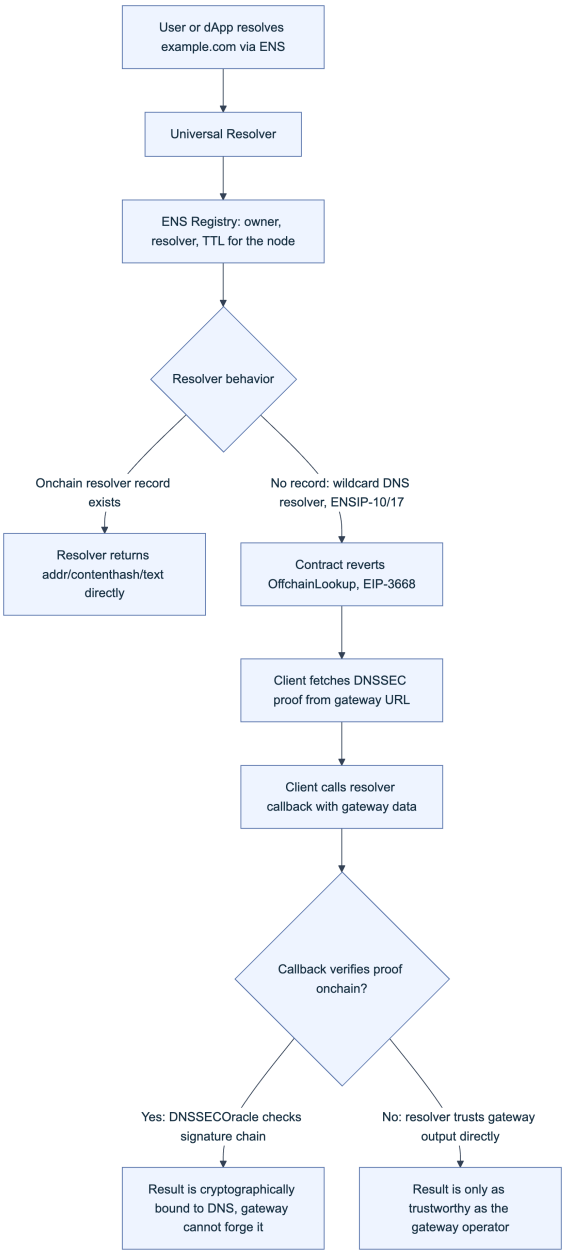

Figure 6. Gasless DNSSEC resolution through CCIP Read. The gateway is a data-fetch convenience; the on-chain callback verification, when present, is what actually establishes trust.

7.3 Cross-Chain and Interface Compliance

ENS resolves addresses for chains other than Ethereum mainnet, and getting that resolution wrong sends funds somewhere unrecoverable. ENSIP-9 added multichain address resolution keyed by SLIP-44 coin type, and ENSIP-11 extended it to EVM-compatible chains by deriving a coin type as the chain ID bitwise-OR'd with `0x80000000`, reversible as `coinType & 0x7fffffff`. A wallet or dApp that miscalculates this mapping, for example confusing a chain ID with a legacy SLIP-44 coin type, can derive or display the wrong address for the same name on a different network.

ENSIP-19 (Multichain Primary Names) extends reverse resolution the same way: a chain-specific reverse namespace lets an address on an L2 have its own primary name, verified by resolving the candidate name forward and confirming it points back to the original address. The specification is explicit that clients must not fall back to a mainnet default name when a chain-specific one is absent, because doing so can present a name that looks correct while actually pointing a transfer at an address the recipient does not control on that chain, which is exactly why ENSIP-19 requires clients to verify chain-specific resolution rather than defaulting.

Interface compliance closes the gap between "the resolver responds" and "the resolver responds correctly." ENSIP-8 (Interface Discovery) lets a resolver point callers at other contracts that implement a given ERC-165 interface, and ENSIP-10's `IExtendedResolver.resolve()` method should itself be interface-discoverable so a caller knows whether wildcard and CCIP-Read behavior apply before it calls in blind. The Universal Resolver, deployed at the vanity address `0xeEeEEEeE14D718C2B47D9923Deab1335E144EeEe`, exists precisely to absorb this heterogeneity: it is the canonical entry point that handles ENSIP-10 wildcard dispatch, EIP-3668 batch gateway calls, and interface checks so individual applications do not each reimplement resolver compliance logic. Basenames, Coinbase's naming layer on the Base L2, is a production example of this cross-chain model: it is "built on the same technology powering ENS names, and deployed on Base," inheriting the same resolver-compliance and offchain-lookup trust questions this section raises.

7.4 Smart Contract and Resolver Risks

Registry ownership gets the attention, but the resolver is where most real compromises land, because it is the contract that actually answers every `addr()`, `contenthash()`, and `text()` call an application makes. Four risk patterns recur across resolver implementations, and each has a different failure mode:

Resolver pattern	Trust model	Verified on-chain?	Failure mode
Standard `PublicResolver`, name owner sets records directly	Owner-controlled	Yes, by registry owner check	Owner key or approved operator compromise rewrites records
CCIP-Read resolver with on-chain proof verification (for example, the DNSSEC oracle path)	Cryptographically verified	Yes, in the callback	Gateway can censor/delay, cannot forge
CCIP-Read resolver that trusts gateway output directly	Gateway-operator-controlled	No	Compromised or malicious gateway operator returns arbitrary data
Custom or unaudited resolver deployed by an integrator	Whatever the contract author built	Unknown until reviewed	Logic bugs, missing access control, or silent reverts break resolution for every name pointed at it

Anyone can deploy a resolver and point their name at it, which means the ENS protocol itself does not guarantee resolver quality; that responsibility sits with whoever set the resolver for a given node (ENS Docs: Resolvers). Approval delegation compounds the risk in the same way NFT-drainer phishing does elsewhere: an owner who signs a `setApprovalForAll` for the base registrar or NameWrapper hands the approved address full control over records and, depending on fuse settings, over transfers, without giving up the underlying private key. Section 8 walks through how these primitives combine into concrete takeovers.

---

8. ENS Threat Model

ENS's threat model splits along the same line as its architecture: attacks on who owns a name, and attacks on what a name resolves to. Both can be executed without ever touching the victim's private key directly.

8.1 Subdomain and Name Takeover

The NameWrapper wraps names as ERC-1155 tokens and introduces fuses: burnable permission bits that permanently revoke a capability. Fuses come in three groups: parent-controlled fuses that only the parent owner can burn (for example, burning `CAN_EXTEND_EXPIRY` lets a subname owner renew independently), owner-controlled fuses that either party can burn to give something up (`CANNOT_TRANSFER` blocks the wrapped token from moving), and subname fuses the parent sets at creation time to define what the child owner actually gets (ENS Docs: Name Wrapper). The critical one for takeover analysis is `PARENT_CANNOT_CONTROL`: until it is burned, the parent name's owner can reclaim, reassign, or delete any subdomain it issued, no matter who currently "owns" that subdomain's token.

That single fact explains most subdomain-takeover incidents. Protocols that hand out `user.protocol.eth` style subdomains without wrapping the child and burning `PARENT_CANNOT_CONTROL` have, whether they intend to or not, kept a master key: a single compromise of the parent name's owner key or an approved operator on that parent yields control of every subdomain issued under it, in one transaction. Legacy unwrapped subdomains created with `setSubnodeOwner` carry the same property by default, since there is no fuse mechanism to burn at all.

Actor	Vector	Impact
External attacker	Phishes a `setApprovalForAll` signature on the base registrar or NameWrapper	Full control of the approved name(s): resolver, subdomains, transfer
External attacker	Front-runs or snipes a `.eth` name that lapsed into expiry and grace period without renewal	Permanent loss of the name to the new registrant; no protocol-level recourse

Actor	Vector	Impact
Semi-trusted integrator	Issues bulk subdomains without wrapping + burning `PARENT_CANNOT_CONTROL`	Parent-key compromise cascades into every subdomain simultaneously
Malicious or compromised parent-name holder	Reclaims a subdomain it issued, even one a legitimate user believed they owned	Silent loss of a user-facing identity (e.g., `alice.dao.eth`) with no user-side signal

8.2 Resolver and Content Manipulation

Once an attacker controls a resolver, whether through a stolen owner key, an abused approval, or a resolver contract with a logic flaw, every record type becomes an attack surface. Rewriting `addr()` redirects payments. Rewriting `contenthash()` swaps the IPFS or Swarm content a front end serves, handing the front-end-hijack playbook from Handbook 01 an ENS-native delivery vector alongside the DNS and CDN ones already covered there. Rewriting text records such as `url`, `avatar`, or social handles turns the name into a phishing lure that still resolves through a name the victim has learned to trust. Rewriting the reverse record, the primary name an address advertises, is display-only and carries no on-chain authorization weight, but it is exactly what wallets and block explorers show next to an address, so an attacker who sets a trustworthy-looking primary name on a scam address can make that address look legitimate at the exact moment a user is deciding whether to sign.

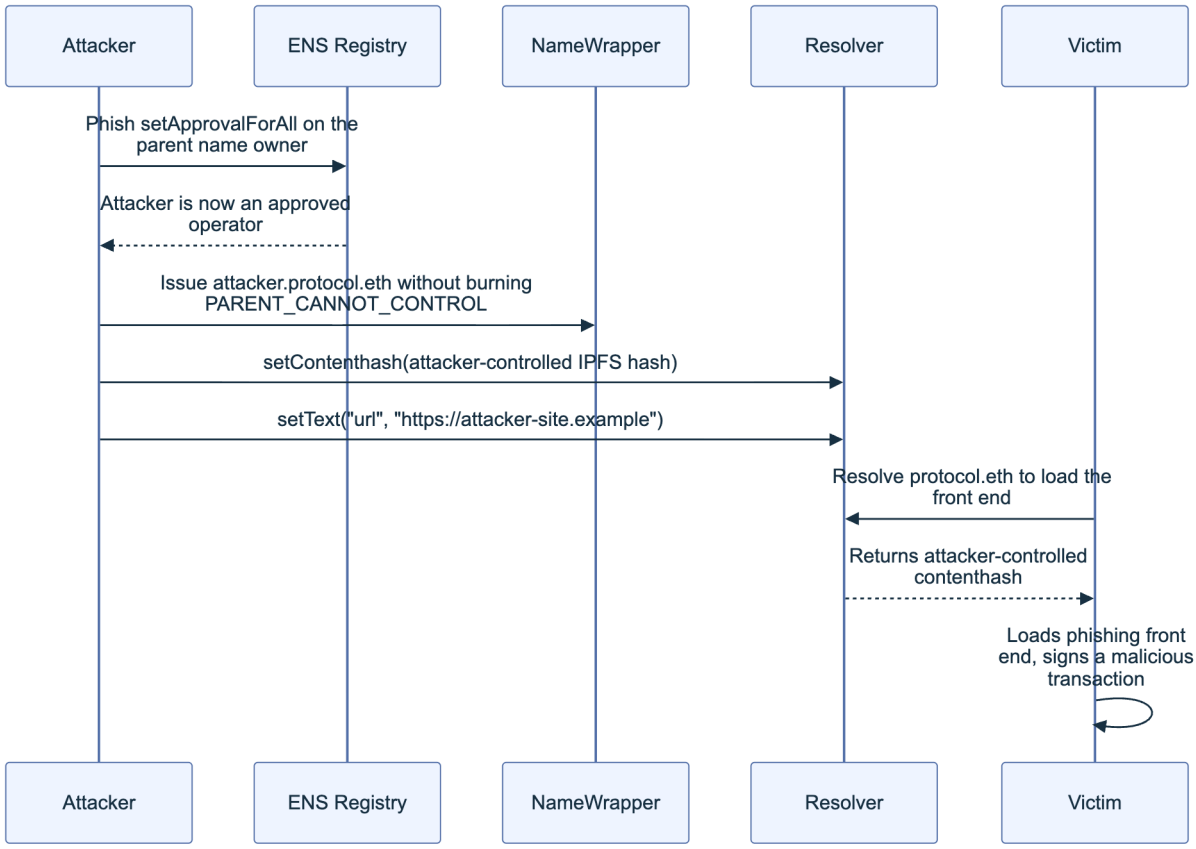

Figure 7. A resolver hijack chain: an approval phish becomes a subdomain and content takeover, which becomes a signed transaction the victim believes came from a trusted name.

9. Playbook: ENS Name or Resolver Compromise

This playbook specializes the general incident-response process in the Incident Response Handbook (06) for the moment a name, resolver, or subdomain under your control starts behaving unexpectedly.

9.1 Detection and Verification

Detection has to happen independently of your own front end, because a compromised resolver will happily lie to a compromised or naive client asking it for confirmation. Query the Registry and resolver directly against a public RPC endpoint and cross-check against a block explorer you did not build.

Detection checklist:

- ☐ Confirm current `owner(node)` on the Registry matches your known-good address; a mismatch means the ownership token itself moved.
- ☐ Confirm current `resolver(node)` matches the resolver you deployed or intended; a mismatch means someone repointed the name.
- ☐ Diff `addr()`, `contenthash()`, and all `text()` keys you use against your last known-good snapshot.
- ☐ If wrapped, call `getData(tokenId)` on the NameWrapper and confirm the fuse bitmap still matches what you set; an unexpected fuse state can indicate a parent-level or approval-level compromise.
- ☐ Check `expires`/grace-period status on the base registrar; a name in grace period is one renewal transaction away from becoming public again.
- ☐ Enumerate current approved operators (`isApprovedForAll`) on both the base registrar and the NameWrapper for the affected name; revoke anything unrecognized (Section 9.2).
- ☐ Pull recent `Transfer`, `NewOwner`, `NewResolver`, `AddrChanged`, `ContenthashChanged`, and `TextChanged` events for the node from an independent RPC or the ENS subgraph to establish a timeline.

```
# Independent verification against a public RPC, not your own infrastructure. {.after-
        printable-checklist}
cast call "$ENS_REGISTRY" "resolver(bytes32)(address)" "$NODE" --rpc-url "$PUB-
        LIC_RPC_URL"
cast call "$RESOLVER" "addr(bytes32)(address)" "$NODE" --rpc-url "$PUBLIC_RPC_URL"
cast call "$RESOLVER" "contenthash(bytes32)(bytes)" "$NODE" --rpc-url "$PUBLIC_RPC_URL"
cast call "$NAME_WRAPPER" "isApprovedForAll(address,address)(bool)" "$OWNER" "$SUS-
        PECT_OPERATOR" --rpc-url "$PUBLIC_RPC_URL"
```

9.2 Recovery and Re-Registration

The recovery path depends entirely on which layer was compromised, and the honest first branch is that some compromises have no protocol-level reversal. ENS ownership is a bearer asset: if the private key holding the name's token, or the token itself, has moved to an attacker, there is no registrar support desk, no dispute process, and no on-chain undo. That asymmetry is the strongest argument for the preventive controls in Section 10 (multisig or hardware-backed ownership, fuses burned on anything you don't want reclaimed, wrapped names for anything security-sensitive) rather than relying on reactive recovery.

If the compromise is a stolen or phished operator approval rather than the underlying owner key, recovery is direct: call `setApprovalForAll(attackerAddress, false)` from the legitimate owner key on both the base registrar and the NameWrapper, then reset every resolver record to known-good values and, if practical, migrate to a fresh resolver contract. If the resolver contract itself is the problem (a logic bug or a gateway-trust model you no longer accept per Section 7.4), point the Registry at a new, reviewed resolver rather than patching in place. If the name lapsed into expiry and was re-registered by a third party, there is no reclamation mechanism comparable to a Uniform Domain-Name Dispute-Resolution Policy (UDRP) process in traditional DNS; your options are negotiating a purchase from the new holder or publicizing a new canonical name across every channel your users trust.

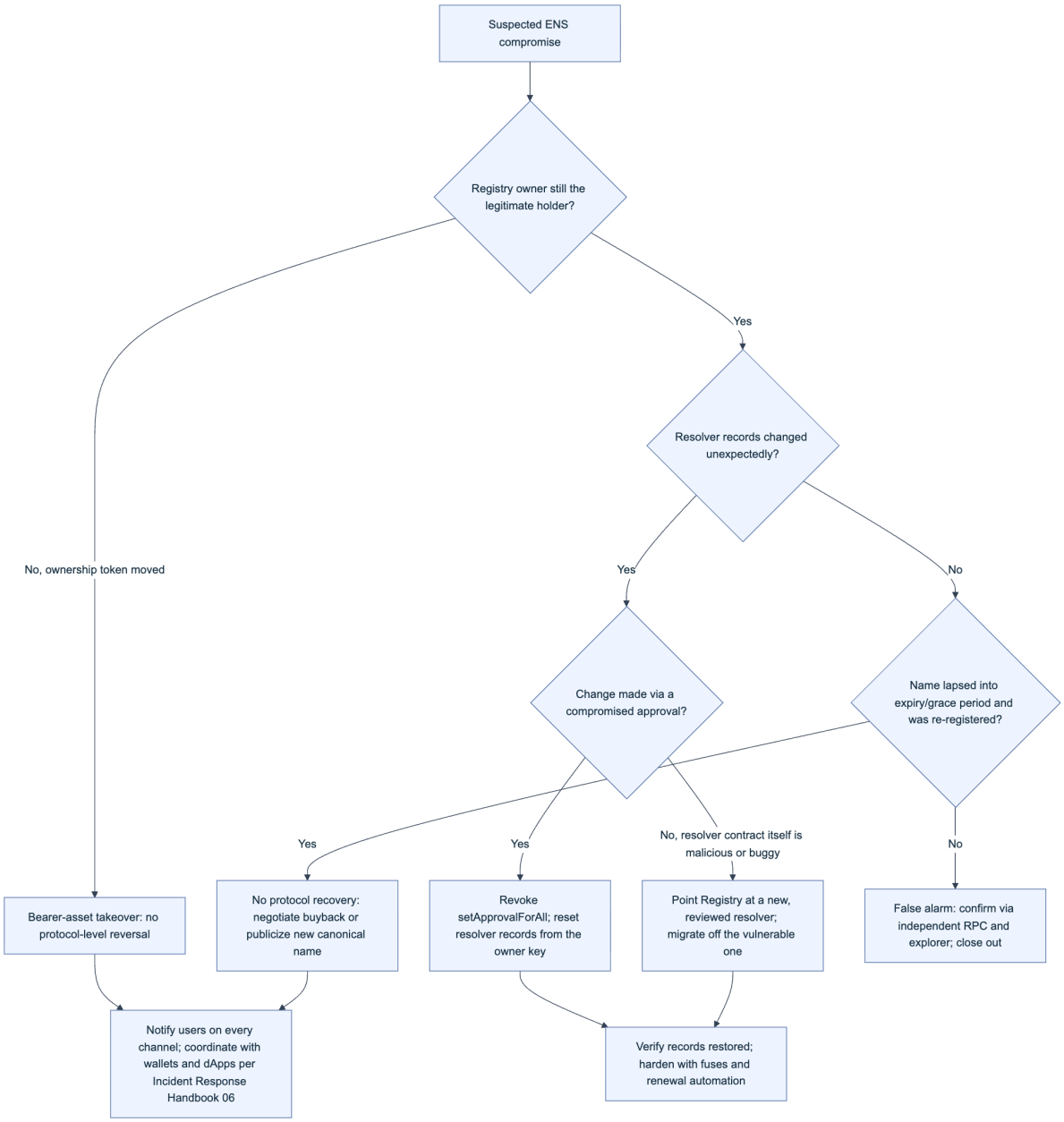

Figure 8. ENS incident triage. The first branch determines whether recovery is even possible at the protocol level, which is why prevention (Section 10) carries more weight here than in most infrastructure playbooks.

10. Best Practices for ENS Integration

The controls below sit at the application layer: what a dApp, wallet, or backend service should do every time it turns user input into a resolved ENS record or turns a resolved record into something rendered on screen.

10.1 Name Handling and Validation

Never resolve or register a name without first running it through ENSIP-15 name normalization, implemented in the reference `ens-normalize.js` library. Normalization strips zero-width characters, enforces a fixed Unicode version, and applies whole-script confusable detection so that a label using, for example, Cyrillic characters that render nearly identically to Latin ones is rejected rather than silently accepted as a different, colliding name (ENS blog: How ENS Normalization Works). Reject non-normalized input outright rather than auto-correcting it; auto-correction can silently resolve a user's typed name to a different node than the one they saw on screen.

```
// Always normalize before resolving or displaying an ENS name.
import { normalize } from '@adraffy/ens-normalize';

function safeResolveInput(rawInput) {
  let name;
  try {
    name = normalize(rawInput); // throws on disallowed/confusable input
  } catch (err) {
    throw new Error('Name failed ENSIP-15 normalization; refusing to resolve');
  }
  return name;
}
```

Beyond normalization, treat every resolver as untrusted until it proves otherwise: check `supportsInterface` before calling `resolve()` on a name that might route through ENSIP-10 wildcard or CCIP-Read logic, and never treat a reverse (primary) name as an authorization signal, since any address holder can set an arbitrary primary name that has no bearing on whether that address should be trusted (Section 8.2). Re-resolve on every session rather than caching indefinitely; ownership, resolver, and fuse state can all change between one user visit and the next.

Validation checklist:

- ☐ Run all user-supplied names through ENSIP-15 normalization before any resolution or registration call.
- ☐ Reject, don't silently correct, names that fail normalization.
- ☐ Verify resolver interface support before calling extended or CCIP-Read resolution paths.
- ☐ Treat reverse/primary names as display hints only, never as access control.
- ☐ Re-resolve rather than cache across sessions for anything security-relevant.

10.2 Referral and Forwarding Security

Two integration patterns deserve their own checks because they route trust decisions through data the ENS protocol itself does not authenticate. First, a growing number of registration front ends, wallets, and subname-issuing platforms attribute `.eth` registrations and renewals to whichever integrator originated them, for revenue sharing or attribution. Wherever your integration passes or receives such an attribution parameter, treat it as untrusted input to be logged and sanity-checked, never as an implicit permission or identity signal; it routes fee accounting, not access control, and should never gate a state-changing action on its own.

Second, "forwarding" through ENS records is common: a `contenthash` record points a subdomain at IPFS or Swarm content, and some integrations additionally use a `url` text record to redirect a visit to a conventional website. Validate the scheme and host of any URL-based forwarding target before navigating a user to it; an unchecked `url` text record is an open-redirect vector sourced from a decentralized record instead of a query-string parameter, and it deserves the same scrutiny as any other redirect target. Prefer content-addressed targets, a `contenthash` pointing at a specific IPFS content identifier (CID), over arbitrary HTTP(S) URLs, since a CID is self-verifying in the way a bare URL is not; this is the same principle behind the Subresource Integrity controls in the CDN and Front-End Supply Chain Handbook (01), applied to a different record type. Finally, if your resolution path invokes CCIP-Read (Section 7.2), remember that the gateway URLs a resolver returns are outside your trust boundary by default; EIP-3668 itself recommends disabling CCIP Read for the specific case of preparing a transaction unless the response is independently verified on-chain (EIP-3668).

Key controls for Part 3

- Verify Registry owner and resolver state against an independent RPC and block explorer, never only your own front end.
- Wrap security-sensitive subdomains with the NameWrapper and burn `PARENT_CANNOT_CONTROL` so a parent-key compromise cannot cascade.
- Judge every CCIP-Read resolver by whether its callback cryptographically verifies gateway data or merely trusts it.
- Verify chain-specific resolution under ENSIP-19 instead of falling back to a mainnet default name for cross-chain transfers.
- Run all user-supplied names through ENSIP-15 normalization and reject failures outright.
- Treat reverse (primary) names, referral parameters, and `url` text records as display or attribution data only, never as authorization.
- Renew `.eth` names before grace-period expiry; there is no protocol-level recovery once a name is re-registered by someone else.

Part IV: IPFS and Decentralized Hosting

Many Web3 teams move their front end onto the InterPlanetary File System (IPFS) to escape a single DNS registrar or CDN account as a point of failure, then discover that decentralized hosting trades one trust problem for three others: which gateway resolved the request, who is still pinning the content, and whether anyone can be compelled to take it down. This part explains how IPFS content addressing actually secures (and fails to secure) a dApp front end, builds a threat model around the gateway and pinning layers where most real incidents occur, and closes with an incident playbook and a hardening checklist.

11. IPFS Security Overview

IPFS is a peer-to-peer, content-addressed storage and distribution protocol: instead of asking "give me whatever is at this location" (the HTTP model), a client asks "give me the content whose hash is exactly this" (IPFS docs: content addressing). That single design choice is the source of both IPFS's security value and its most common Web3 deployment mistakes, which the next two sections unpack before Chapter 12 turns them into a threat model.

11.1 Content Addressing and Integrity

Every object stored on IPFS is identified by a Content Identifier (CID), a self-describing label built from three components under the multiformats family: a **multihash** (the cryptographic digest, SHA-256 by default), a **multicodec** (how to interpret the retrieved bytes) and a **multibase** (how the whole thing is text-encoded). Change one byte of the underlying content and the digest changes, so the CID changes. This is the property teams reach for when they say "IPFS gives us integrity by default": a CID is not a location, it is a claim about content, and any node that returns bytes not matching that claim has failed verification, full stop (IPFS docs: content addressing).

Two details trip up engineers new to the model. First, a CID does **not** equal a checksum of the raw file. When you add a file to IPFS it is chunked (commonly into 256 KiB blocks), organized into a Merkle DAG (Directed Acyclic Graph), and the CID hashes the root of that DAG, not the flat file bytes; running `sha256sum` on the original file will not match the multihash inside the CID.

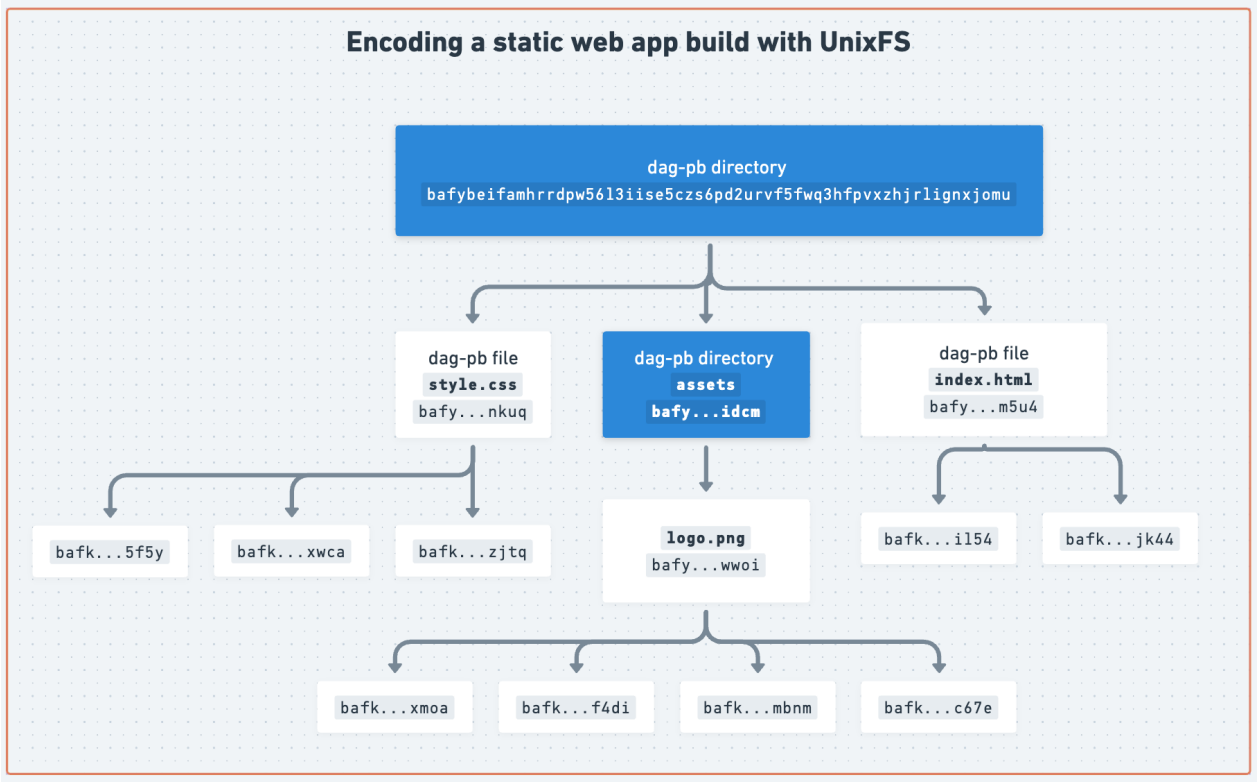

Figure. A concrete UnixFS DAG rather than an abstract binary tree. Large file blocks sit at the leaves, intermediate nodes encode links to those blocks, and the root CID commits transitively to every link and chunk. Investigators must retain the whole reachable DAG: preserving only the root block does not preserve the file it names. Source: IPFS Documentation, Content Lifecycle, documentation visual licensed under the IPFS documentation project's open-source terms.

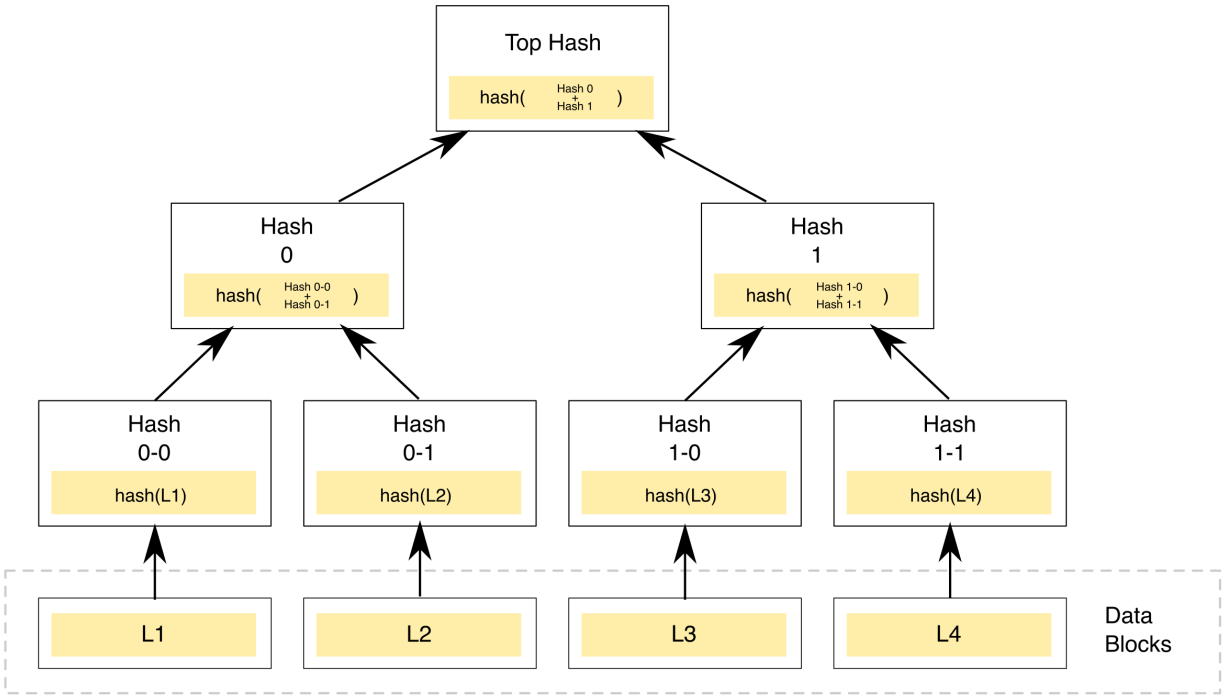

Figure. The general hash-tree construction behind a Merkle DAG. IPFS's CID is functionally the "Top Hash" of a tree built this way over a file's chunks, which is why changing any single leaf block changes every hash above it, all the way up to the CID itself. Source: Wikimedia Commons, CC0 1.0 (public domain).

Second, there are two CID versions in active use. **CIDv0** is a fixed Base58 encoding that always starts with `Qm` (46 characters). **CIDv1** adds an explicit multibase prefix and multicodec, and is required for subdomain-based gateway hosting because it can be encoded as a DNS-label-safe string (IPFS docs: CID versions). A front end pinned under a CIDv0 identifier cannot be served with per-origin isolation on a subdomain gateway without first converting it to CIDv1; treat that conversion as a deployment step, not an afterthought.

```
CID anatomy (CIDv1, base32):
  bafybeigdyrzt5sfp7udm7hu76uh7y26nf3efuylqabf3oclgtqy55fbzdi
  └┘└┘└──────────────────────────────────────────────────────┘
   │  │                        │
   │  │                        digest bytes (multihash payload)
   │  multicodec + multihash function id, multibase-encoded
   multibase prefix ('b' = base32)
```

Content addressing gives you a verification primitive, not automatic security. A malicious file has a perfectly valid CID; IPFS proves the bytes you received match the CID you requested, it says nothing about whether the CID you requested was the right one. That distinction is why Section 11.2 and Chapter 12 exist: the CID is only as trustworthy as the channel that told you what CID to fetch.

11.2 Gateway and Pinning Trust

Most users and most dApp front ends never run a full IPFS node. They reach content through an **HTTP gateway**, a bridge process that speaks IPFS on one side and plain HTTPS on the other, and they rely on a **pinning service** to keep the content available on the network at all. Both are trust delegations, and both differ sharply from the CDN model covered in Part 2 of this handbook.

Gateways come in three resolution styles, per the IPFS HTTP Gateway specification:

Style	URL shape	Origin isolation	Recommended for
Path gateway	`https://gateway.example/ipfs/{CID}/index.html`	None, all CIDs share the gateway's origin	Quick fetches, never for hosting an app
Subdomain gateway	`https://{CID}.ipfs.gateway.example/index.html`	Full, each CID gets its own browser origin	Hosting a dApp front end
DNSLink gateway	`https://app.example/` (via `_dnslink.app.example` TXT record)	Depends on the DNS record's target CID	Human-readable production URLs

Path-style gateways violate the same-origin policy: two unrelated pieces of content served under `/ipfs/CID-A/` and `/ipfs/CID-B/` share one browser origin, so a malicious CID can read cookies, local storage, or DOM state left behind by a legitimate one hosted on the same public gateway (IPFS docs: IPFS gateway). This is not a theoretical footnote; it is the reason the specification recommends subdomain gateways for anything that runs application code.

The deeper trust question is what a gateway actually guarantees. A **recursive** public gateway (ipfs.io, dweb.link, and similar) fetches content on your behalf from the wider network and hands you the result over ordinary HTTPS: you are trusting that operator's honesty and its TLS termination, exactly as you would trust any web host, unless you separately verify what it returned. A **trustless** gateway response, by contrast, can be requested in CAR (Content Addressable aRchive) format: the client receives the raw DAG blocks and hashes them itself, so a dishonest or compromised gateway cannot substitute content without the client detecting the mismatch. Few dApp front ends implement this verification themselves; most simply point an `<img>` or `<script>` tag at a gateway URL and trust it, which reintroduces exactly the "trust the host forever" failure mode Part 2 spent an entire chapter eliminating for CDNs.

Pinning is the second, orthogonal trust delegation. IPFS nodes garbage-collect data they are not explicitly told to retain, so content that nobody pins can silently vanish from the network the moment the last node holding it goes offline: "IPFS guarantees that any content on the network is discoverable, it doesn't guarantee that any content is persistently available" (IPFS docs: persistence, pinning, and garbage collection). Commercial pinning services (Pinata, Filebase, Storacha Network, formerly web3.storage) exist precisely to hold that guarantee on your behalf,

typically backed by storage deals on the Filecoin network. That guarantee is a business relationship with its own single point of failure: an expired card, a terminated account, or a shut-down service silently converts your “decentralized” front end into a 404.

12. Threat Model for IPFS

The failure modes below are not exotic. Every one has a documented real-world instance or a standing, actively exploited pattern. Model them the same way Part 1 modeled the CDN and dependency surfaces: by actor, by entry point, and by the control that closes it.

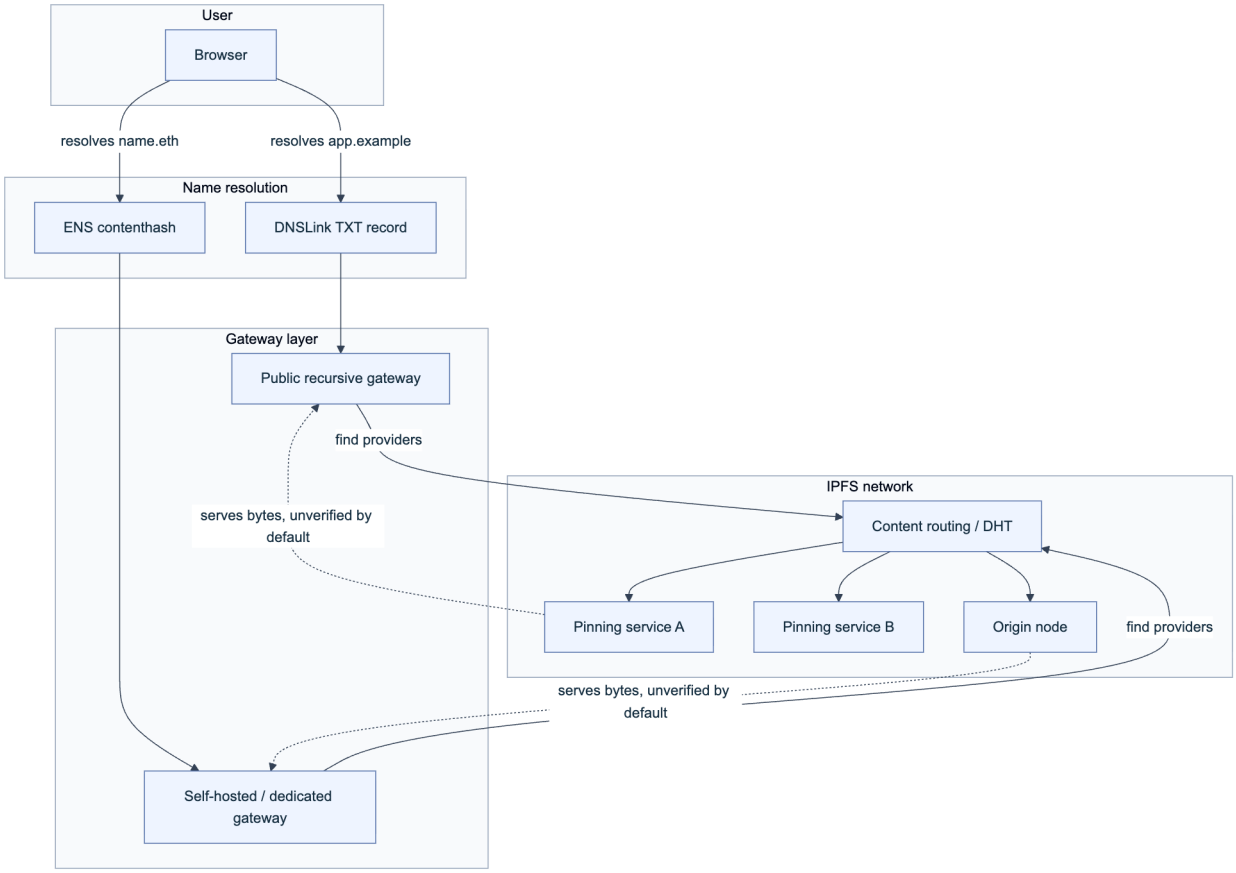

Figure 1. The IPFS resolution path for a Web3 front end. Each arrow is a place where the wrong answer can be substituted: a poisoned DNSLink record, a compromised gateway, or a pinning provider that has quietly stopped serving.

12.1 Gateway Compromise and MITM

A gateway sits exactly where a CDN edge node sits in Part 2’s model, and it inherits the same power: it can read every request and, if operating as a recursive (non-verifying) gateway, rewrite every response before it reaches the browser. Three concrete failure paths matter here.

Operator-level compromise. If an attacker gains control of a public or dedicated gateway’s infrastructure (compromised credentials, a malicious insider, or a coerced operator), they can serve substituted bytes for a requested CID as long as the client is not independently verifying the CAR

response. This is architecturally identical to a CDN compromise, with one added wrinkle: because most gateways are third-party public infrastructure rather than something the protocol team provisions and monitors, detection is even less likely.

DNSLink and ENS pointer hijack. A production dApp usually resolves a human-readable name to a CID through a DNSLink TXT record (`_dnslink.app.example` → `/ipfs/CID`) or an Ethereum Name Service (ENS) `contenthash` record defined by EIP-1577, which lets an ENS name resolve directly to IPFS, IPNS (InterPlanetary Name System, IPFS's mutable-pointer scheme), or Swarm content instead of a traditional IP address. Both are pointers, and a pointer is only as safe as who can write to it. A DNSLink record is exactly as vulnerable to registrar or DNS-provider hijack as the A/AAAA record hijacks covered in Part 2 of this handbook; the OWASP Web3 Attack Vectors Top 15 tracks this class as **WA13, DNS, Domain and Routing Hijacking**. An ENS `contenthash` record is only as safe as the private key or multisig that controls the name; if that key is phished or the resolver's owner account is compromised, an attacker republishes the same human-readable name against a CID they control, and every wallet extension and browser that resolves `name.eth` serves the attacker's front end with the domain's full apparent legitimacy intact.

Malicious or squatted CID served as "the app." Because CIDs are simply strings, a phishing operator can host a pixel-perfect clone of a dApp's UI under their own CID and their own gateway subdomain, then distribute the link through search ads, Discord, or Twitter/X. The IPFS network never distinguishes "this is the real app" from "this is a plausible clone"; only the DNSLink or ENS name binding does that. Treat name-to-CID resolution as the actual trust boundary, not the IPFS layer beneath it.

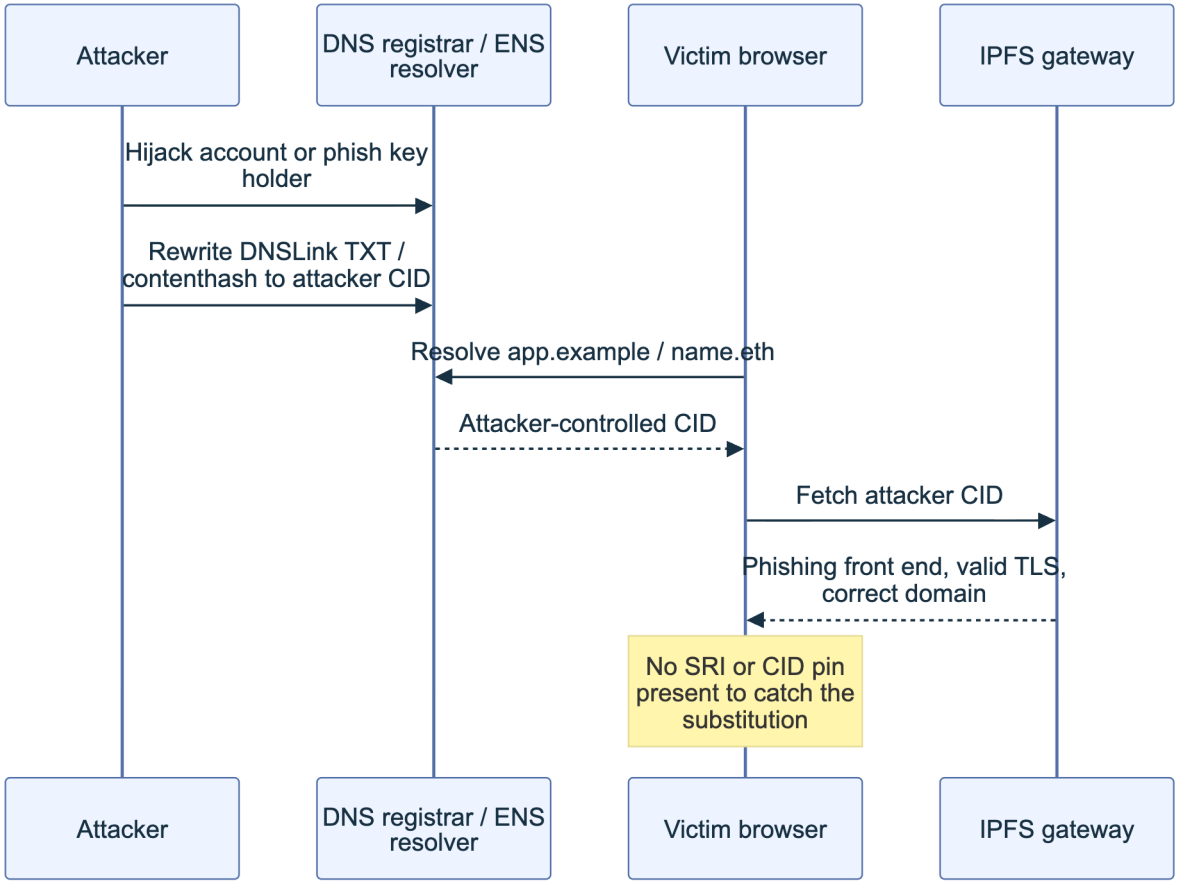

Figure 2. Gateway compromise is rarely needed when the pointer (DNSLink or ENS contenthash) can be hijacked directly. The user's browser shows a correct domain and valid certificate throughout.

This class of attack has hit production dApps repeatedly through the DNS half of the equation covered in Part 2; the IPFS-specific variant substitutes an ENS resolver or DNSLink write for a registrar account takeover, but the endgame, a front end that requests malicious signatures under the real domain, is identical.

12.2 Pinning and Availability Abuse

Availability is a security property for a dApp front end, because if the only path to a user's funds goes dark, the practical effect on the user is indistinguishable from a successful attack. Two distinct failure modes live here.

Silent unpinning. If your deployment pipeline pins a build to exactly one pinning service and that service terminates the account (non-payment, policy violation, outage, or business shut-down), the content is garbage-collected off that provider's nodes on its normal schedule. Nothing announces this; the CID simply stops resolving once no node on the network still holds it. Commercial pinning has undergone real consolidation: Protocol Labs' web3.storage service

migrated and rebranded to Storacha Network, built around the `w3up` protocol stack, and providers change terms and free-tier availability on their own schedule. A single-provider pin is a single point of failure with the same blast radius as a single-registrar domain.

Public gateway abuse and rate limiting. Public recursive gateways are expensive to run and are a favorite target for anonymous bulk retrieval, including malware and phishing kit distribution (Section 12.3). Gateway operators respond with aggressive rate limiting, CAPTCHA challenges, or outright blocking of ranges and CIDs, and legitimate dApp front ends that depend on a shared public gateway inherit that operator's abuse-response posture. A front end hardcoded to `https://ipfs.io/ipfs/{CID}` degrades for every user the moment that specific gateway throttles the request pattern your app happens to generate, even though your content is not the abusive traffic.

DHT-level denial and eclipse risk. Content discovery on the public IPFS network runs over a Kademlia-style Distributed Hash Table (DHT) that peers use to find which nodes currently hold a given CID's blocks (IPFS docs: nodes and network layer). Kademlia-family DHTs are documented in the academic literature as vulnerable to Sybil attacks (an attacker floods the routing table with nodes they control) and eclipse attacks (an attacker surrounds a target node with malicious peers, isolating it from honest ones), the same class of routing-layer attack studied for BitTorrent's Mainline DHT and other Kademlia deployments. For a Web3 team, the practical mitigation is not "fix the DHT," it is "do not depend on the public DHT alone to make your production front end reachable": pin to multiple independent providers and, for anything traffic-sensitive, front the content with a dedicated gateway you operate rather than relying purely on ad hoc peer discovery.

Availability failure	Root cause	Primary control
Silent unpinning	Single pinning provider, account lapse or shutdown	Pin to 2+ independent services; monitor pin status
Public gateway throttling	Shared abuse-prone infrastructure	Operate or pay for a dedicated gateway; multi-gateway fallback
DHT Sybil / eclipse	Kademlia routing layer has no default Sybil resistance	Do not rely on DHT discovery alone for production traffic
Provider business exit	Startup or product shutdown (rebrand, sunset, acquisition)	Contractual SLA, own a Filecoin storage deal directly, re-pin on notice

12.3 Content and Censorship Considerations

IPFS's resistance to takedown is a double-edged property, and Web3 teams need to threat-model both edges. On one side, the same persistence that protects a legitimate dApp front end from a single DNS or hosting takedown also protects malicious content. Cisco Talos has documented sustained abuse of IPFS gateways for phishing kit hosting (fake Microsoft and DocuSign login pages reached through spoofed PDF lures) and as a payload delivery channel for malware families such

as Agent Tesla downloaders, precisely because “anyone can set up an IPFS gateway” and takedown of one gateway or one pin does not remove the content from the wider network as long as any other node retains it. The consequence for your protocol: enterprise security stacks increasingly rate-limit or block IPFS gateway domains outright at the network perimeter as a blanket anti-abuse measure, which can degrade your legitimate front end for users behind those controls with no action on your part.

On the other side, “censorship-resistant” is a claim about the protocol layer, not about every implementation detail of a given deployment. A dApp that resolves through a single ENS name controlled by one signer, backed by pins held at one commercial provider, and served predominantly through one public gateway has recreated a single point of legal and operational pressure at each layer, even though the underlying storage protocol is peer-to-peer. Gateway operators are commercial entities in identifiable jurisdictions that do receive and act on takedown requests for specific CIDs; a pinning provider can be compelled to stop serving a customer; an ENS name’s controlling key can be subpoenaed or, if custodial, frozen. None of this makes IPFS a poor choice, it means resilience has to be engineered deliberately across gateways, pinning providers, and name-resolution paths rather than assumed from the protocol’s marketing.

Design for graceful degradation rather than an illusion of unstoppability: publish the raw CID prominently (GitHub repo, ENS text record, social channels) so a user whose primary gateway blocks or censors a specific CID can retrieve identical content through any other gateway or a local Kubo node.

---

13. Playbook: IPFS Gateway or Pinning Incident

Treat an IPFS-layer incident with the same discipline as the CDN incident response covered elsewhere in this handbook series, adapted for the specific signals this stack produces. Work the phases in order; do not skip verification before communicating a root cause.

Phase 1: Detect and scope (minutes 0-15). - Confirm the CID currently being served does not match the last known-good CID published in your release records or Git tag. - Check whether the anomaly is at the pointer layer (DNSLink TXT record, ENS `contenthash`) or the content layer (a specific gateway returning wrong bytes for the correct CID). - Pull the DNSLink TXT record directly: `dig TXT _dnslink.app.example +short`. Compare it against your source-controlled deployment manifest. - Query the ENS `contenthash` directly against a resolver you trust (not through the app’s own front end, which may itself be compromised): `bash cast call $(cast resolve-ens name.eth) "contenthash(bytes32)(bytes)" $(cast namehash name.eth) --rpc-url $RPC_URL` - Test at least three independent gateways (a public one, a dedicated/paid one, and a local Kubo node if available) with the *same* CID to isolate whether the problem is one gateway or the pointer itself.

Phase 2: Contain (minutes 15-60). - If the pointer was hijacked: rotate the compromised credential or ENS-controlling key immediately, and if the name is behind a multisig or timelock, trigger the emergency revocation path defined in your incident response runbook (see the Incident Response Handbook, 06). - If a specific gateway is serving substituted content: stop directing users to it. Update any hardcoded gateway URL in your front end's fallback list and push the change through your normal deploy path, not through the compromised path. - Publish an out-of-band notice (status page, verified social account, GitHub release) stating the last-known-good CID in full, so users can bypass any compromised resolution path and fetch the correct content directly by CID.

Phase 3: Eradicate and recover. - Re-pin the known-good CID across every configured pinning provider and confirm pin status via each provider's API, do not assume the pin survived the incident. - If the ENS name or DNSLink zone was compromised, audit every record the attacker could have touched, not only the one that was visibly wrong; contenthash hijacks are frequently paired with resolver-level changes that persist after the obvious symptom is fixed. - Rotate all keys with write access to the ENS name, the DNS zone, and every pinning service account involved.

Phase 4: Post-incident. - Document the exact pointer chain (registrar → DNS/ENS → gateway → pin) that failed and which link broke. - Add the failed pointer to automated monitoring (Chapter 14) so the same class of failure alerts before a user reports it. - If the incident involved phishing content served under your name, file takedown requests with the specific gateway operators serving it and, where applicable, report the abusive CID to the pinning provider hosting it.

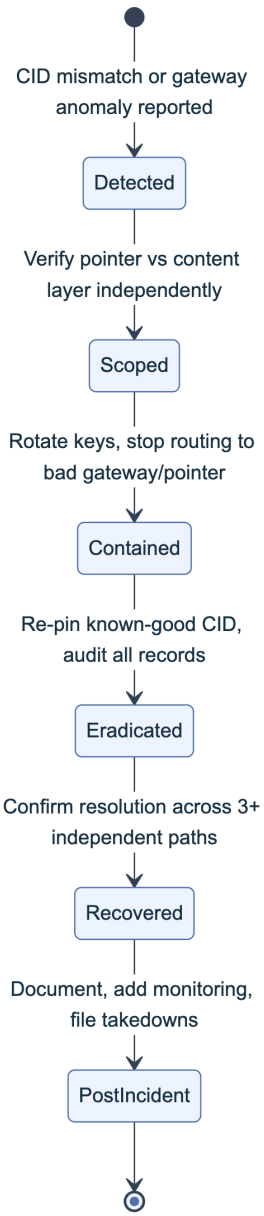

Figure 3. Incident lifecycle for an IPFS gateway or pinning compromise. Scoping happens before containment because the fix for a hijacked pointer and the fix for a compromised gateway are different actions taken by different owners.

14. Best Practices for IPFS in Web3

Consolidate the controls implied across this part into a deployment standard your team can audit against.

- **Pin to at least two independent providers.** Never let a single commercial pinning service be the sole reason your content still exists on the network; treat this the same way Part 2 treats multi-CDN redundancy.

- **Verify, don't trust, gateway responses where it matters.** For anything security-sensitive, request trustless CAR responses and hash-verify locally instead of accepting a recursive gateway's word for it; at minimum, publish the CID alongside a conventional SRI-style hash a user's tooling can cross-check.
- **Use CIDv1 and subdomain gateways for anything that runs code.** Path-style gateway hosting breaks origin isolation; it is acceptable for static data retrieval, never for a front end that signs transactions.
- **Treat the name-to-CID pointer as the actual trust boundary.** Secure the ENS `contenthash` owner key and the DNSLink zone with the same rigor Part 2 and Part 3 of this handbook apply to registrar accounts and ENS ownership: hardware-backed signers, multisig or timelock for changes, and monitoring on every write.
- **Monitor the deployed CID continuously.** Poll your production DNSLink record and ENS `contenthash` on a schedule and alert on any value that does not match your release manifest; this is the IPFS-layer equivalent of certificate transparency monitoring.
- **Publish the canonical CID out of band.** Keep the current CID visible in your GitHub repository, ENS text records, and verified social channels so users have a way to bypass a compromised gateway or DNS path entirely.
- **Plan for gateway blocking as a normal operating condition, not an edge case.** Offer a self-hosted or dedicated gateway option and document how technical users can run a local Kubo node against your CID directly, since enterprise networks increasingly block public IPFS gateway domains wholesale.
- **Back critical pins with a direct storage relationship**, such as a Filecoin storage deal you hold rather than one mediated entirely through a single reseller, so a provider's business decisions cannot unilaterally end your content's availability.

Key controls for Part 4

- Multi-provider pinning with active pin-status monitoring, never a single pinning service as the sole source of availability.
- CIDv1 plus subdomain gateway hosting for any front end that executes code or requests signatures.
- Trustless (CAR-verified) retrieval, or at minimum a published CID/hash, for security-sensitive content instead of blind trust in a recursive gateway.
- Hardware-backed, multisig-protected control of the ENS `contenthash` owner key and the DNSLink DNS zone, with continuous monitoring for unauthorized changes.
- An incident playbook that scopes pointer-layer versus content-layer failure before attempting containment.

Part V: Hosting and Web Presence

DNS and ENS answer "where do I send the user," but something still has to answer at the other end: a virtual private server (VPS), a cloud platform account, an object storage bucket, or a static-hosting provider that holds the actual front-end bundle. This Part treats that hosting layer as its own trust boundary, draws a firm line between it and the CDN and DNS layers covered elsewhere in this series, and gives you a playbook for the moment a front end shows attacker-controlled content instead of your own.

15. Traditional Hosting (VPS, Cloud) and Web3

Most Web3 teams describe their stack as "decentralized" while their marketing site, documentation, and often the dApp front end itself sit on entirely conventional infrastructure: a VPS from a provider like DigitalOcean or Linode, a Platform-as-a-Service (PaaS) such as Vercel or Netlify, or a storage bucket on Amazon Web Services (AWS) S3 or Google Cloud Storage (GCS) fronted by a content delivery network (CDN). None of that is a mistake. Conventional hosting is fast, cheap, and operationally simple compared with fully decentralized alternatives, and Part 4 already covered the trust tradeoffs IPFS makes to avoid it. What matters is that the team treats this layer with the same rigor it applies to a smart contract's access control: someone owns the account, that account has credentials, and whoever holds those credentials controls exactly what code a user's browser executes.

The account-takeover attack pattern here mirrors the registrar compromises in Part 2, only one layer downstream: instead of repointing where a domain resolves, the attacker changes what the destination serves. MITRE ATT&CK T1078.004, Valid Accounts: Cloud Accounts, catalogs exactly this technique, noting that adversaries who obtain legitimate cloud credentials through phishing, credential stuffing, or leaked API keys can establish persistence and modify resources with the

same authority as the legitimate operator, often without triggering any alert that assumes "if the login succeeded, it was the owner." A cloud console session is functionally equivalent to a registrar session or a signing key: whoever holds it, publishes.

15.1 Front-End Hosting and Integrity

Hosting models fall into three broad shapes, and each shifts different responsibilities onto the team. A self-managed VPS gives full control of the operating system, web server, and TLS termination, which means the team owns patching, SSH key hygiene, and firewall configuration directly; a compromised SSH key or an unpatched service is a direct path to arbitrary content substitution. A PaaS such as Vercel, Netlify, or Cloudflare Pages removes server management but concentrates trust in the platform account: whoever can push a deployment, or hold an API token with deploy scope, controls production. Object storage behind a CDN (S3 or GCS behind CloudFront or Cloud CDN) removes the server entirely, but the bucket's access policy becomes the whole security boundary, and a public-write misconfiguration or a leaked service-account key is as good as root.

Two structural properties of PaaS and CI/CD-driven hosting are worth calling out because they change the shape of the threat model rather than just relocating it. First, most of these platforms build from source on every push, so the CI/CD pipeline itself, not only the hosting console, is a place an attacker can inject code; the OWASP Secrets Management Cheat Sheet applies directly here, since a deploy-scoped API token embedded in a workflow file or a poorly scoped CI secret is functionally a master key to the front end. Second, immutable, versioned deployments (each build produces a new, addressable artifact rather than overwriting one mutable "production" target in place) make rollback trivial and give you an audit trail of exactly what was live at what time, an approach with deep roots in Kief Morris's immutable server pattern and directly applicable to how modern PaaS platforms already build and promote deployments.

A quieter failure mode is the dangling DNS record: a team provisions a subdomain that points to a cloud resource (an S3 bucket, a Heroku app, an Azure endpoint), later deprovisions that resource, and forgets to remove the CNAME. Because the DNS record still resolves, an attacker who notices the dangling pointer can claim the same resource name on the same provider and have the subdomain now serve their content, with a valid TLS certificate issuable in minutes because domain-validated certificate authorities only check that the requester controls the (now attacker-owned) resource. The `can-i-take-over-xyz` project catalogs more than one hundred cloud and SaaS providers by exactly this fingerprint, and its "vulnerable" list includes major object storage and static-hosting services when a bucket or app name is deleted without first removing the pointing record.

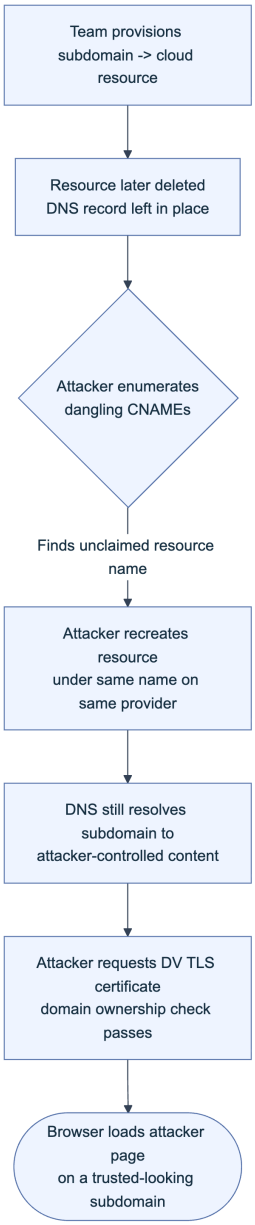

Figure 1. Subdomain takeover through a dangling DNS record. No credential is stolen; the attacker only needs the deprovisioned resource name and a provider account.

Every row in the table below assumes multi-factor authentication (MFA) enforced with a hardware security key, not a one-time code delivered by SMS or a software authenticator an attacker can re-enroll through a support ticket, as the non-negotiable baseline; the differences between hosting models are in what gets layered on top of that baseline.

Hosting model	Who holds the trust boundary	Primary failure mode	Primary control
Self-managed VPS	SSH keys, OS patch level, firewall rules	Leaked key, unpatched service, exposed management port	Key rotation, CIS-benchmarked hardening, no direct-internet SSH

Hosting model	Who holds the trust boundary	Primary failure mode	Primary control
PaaS (Vercel, Netlify, Cloudflare Pages)	Platform account, deploy-scoped tokens, connected Git integration	Stolen account session, over-scoped CI token, unreviewed preview deploy promoted	Hardware-key MFA, least-privilege deploy tokens, branch protection on the deploying branch
Object storage + CDN (S3/GCS + CloudFront)	Bucket/IAM policy, service-account keys	Public-write bucket policy, leaked service-account key, dangling record after deprovisioning	Least-privilege IAM (per the AWS Well-Architected Security Pillar), bucket policy review, DNS inventory hygiene
IPFS-pinned static build	Pin service account, ENS content-hash owner	See Part 4	See Part 4

The common thread across every row is that the hosting account is a signing key in every practical sense: whoever can authenticate to it can publish arbitrary content to every user who trusts your domain. Hardware-key MFA, least-privilege API tokens scoped to a single project rather than the whole organization, and a maintained inventory of every DNS record and the cloud resource it points to are the three controls that close the largest share of this chapter's incidents.

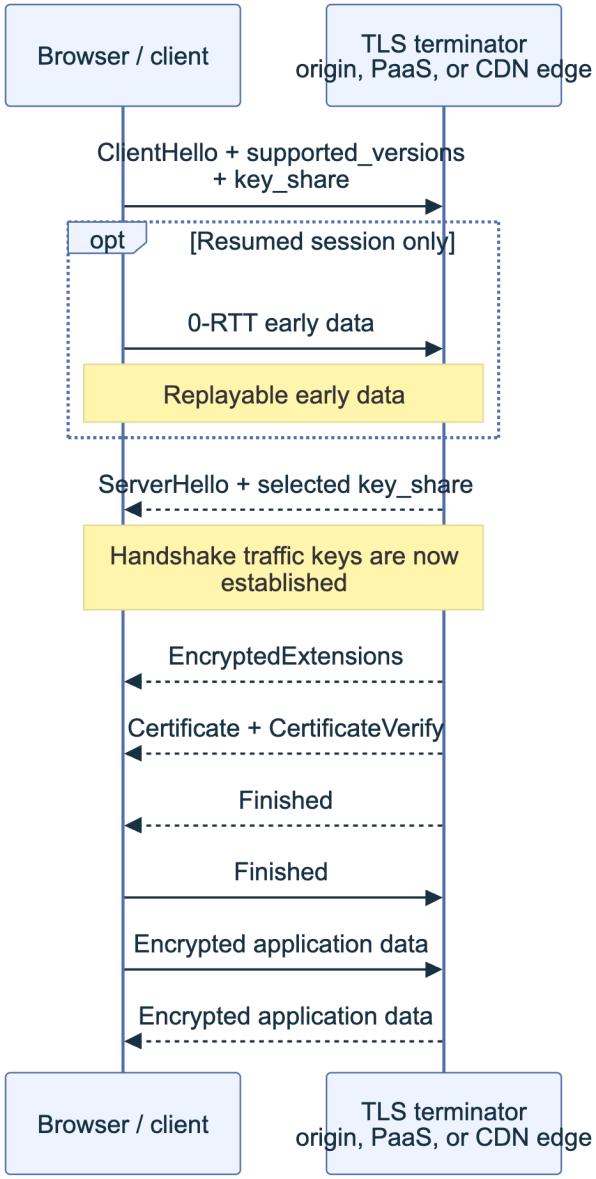

Figure. TLS 1.3 full-handshake flow. Messages after `ServerHello` *are encrypted, the legacy TLS 1.2 key-exchange sequence is gone, and* `ChangeCipherSpec` *is not part of the cryptographic state transition (except as an optional middlebox-compatibility artifact). Whoever controls this termination point controls the certificate and application bytes the browser accepts, which is why TLS 1.3, HSTS, and strict protection of the hosting or CDN account are one control chain.*

15.2 Overlap with CDN and Supply Chain Handbook

This handbook and the CDN and Front-End Supply Chain Security Handbook (01) describe adjacent, easily confused layers, and the boundary is worth stating precisely so a team does not assume the other handbook's controls cover a gap this one leaves open. This handbook's hosting chapter is about **where the origin content lives and who can change it**: the VPS, the PaaS account, the storage bucket. Handbook 01 is about **what happens to that content on its way to and inside the browser**: Subresource Integrity (SRI) pinning, Content Security Policy (CSP), CDN cache behavior, and the build pipeline that produces the artifact in the first place. A hosting

compromise changes the source; a CDN or supply-chain compromise changes or misuses something downstream of a correct source. Both end in the same place, a user's browser executing attacker-chosen code, so both need controls, and neither substitutes for the other.

The BadgerDAO incident of December 2, 2021 shows why the boundary matters in practice rather than only in theory. Attackers had, over a window beginning around November 20, inserted malicious token-approval prompts into BadgerDAO's live front end by compromising the team's Cloudflare account and using Cloudflare Workers, an edge-compute feature that intercepts and can rewrite responses before they reach the browser, to inject a script that requested unlimited spending approval from any wallet that connected. More than 500 addresses granted the approval before the team paused the affected contracts roughly two hours and twenty minutes after mass draining began on December 2, and losses reached approximately $120 million in wrapped Bitcoin and ERC-20 tokens. No VPS was touched and no origin bucket was modified; the compromise lived entirely in the CDN account layer, which is why Handbook 01's SRI and CSP guidance, not this handbook's hosting hardening, is the direct countermeasure (a strict CSP would have blocked the injected script's ability to run, and SRI would have made the tampered response fail a hash check before execution). The lesson for this Part is narrower but still load-bearing: a team that fully hardens its VPS and storage bucket while leaving its CDN account on a shared password with no MFA has hardened one boundary and left the adjacent one, covered by a different handbook, wide open.

Layer	Governing handbook	Representative control	Representative failure
DNS / registrar	This handbook, Part 2	DNSSEC, registrar lock, CAA records	Curve Finance, Aug 2022 (registrar takeover)
Origin hosting (VPS, PaaS, bucket)	This handbook, Part 5	Hardware-key MFA, least-privilege IAM, DNS inventory	Dangling-record subdomain takeover
CDN / edge delivery	Handbook 01, Part II	SRI, CSP, CDN account MFA, cache-key hygiene	BadgerDAO, Dec 2021 (Cloudflare Workers injection)
Build and dependency chain	Handbook 01, Part III	SBOM, signed provenance, pinned CI actions	Polyfill.io, Jun 2024
Broader server/network/secrets	Handbook 07	Network segmentation, secrets management, patch cadence	See Handbook 07

Practically, treat these five rows as a single checklist to walk when scoping a security review, not as five independent teams' problems. A front end is only as trustworthy as the weakest row, and an attacker does not care which handbook's chapter they exploited.

16. Playbook: Front-End Defacement or Compromise

Front-end defacement is the moment a user loads your domain and gets someone else's content: a phishing clone, an injected drainer script, or a static "hacked by" page. The 2024 Squarespace domain migration incident is the clearest recent case study for this playbook because it shows the full shape of the failure: an infrastructure-layer weakness at a domain registrar, not a smart contract bug, not even direct credential theft, produced a live phishing front end on legitimate, previously trusted domains. When Squarespace completed its migration of roughly ten million domains transferred from Google Domains, migrated accounts were created against the registrant's email address but left without a password and, during the transition window, without multi-factor authentication enforced; anyone who arrived at the login flow with the correct email address could set a new password and take the account with no proof of email ownership required. Attackers exploited this starting on July 9, 2024, seizing DNS control of Compound Finance's, Celer Network's, and Pendle Finance's domains among others, and researchers later identified roughly 228 additional DeFi front ends that remained exposed through the same migration pathway. Once in control of DNS, the attackers pointed the domains at cloned interfaces running the Inferno drainer kit, a phishing toolkit that presents a normal-looking wallet-connect flow and requests a signature that empties the connected wallet rather than performing the advertised action.

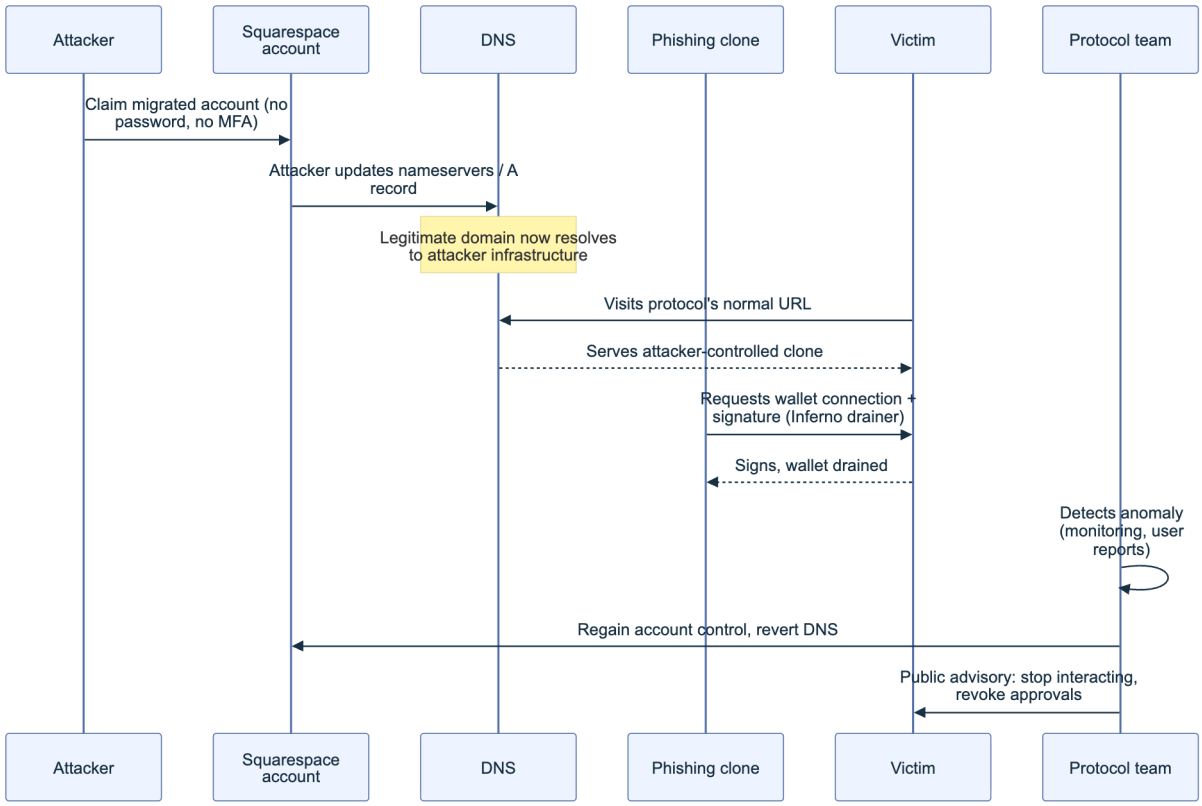

Figure 2. The 2024 Squarespace mass-hijack pattern. No credential was phished from the protocol team; the weakness lived entirely in the registrar's account-migration flow, and the payoff was a drainer kit dropped on a domain users had no reason to distrust.

Namefi, "The 2024 Squarespace DeFi Domain Mass-Hijack"; SC Media coverage via dnsafrica.org.

Compound Finance's main domain was confirmed hijacked; Celer Network and Pendle Finance both recovered their domains and, notably, Pendle publicly confirmed that no user funds were lost because the team caught and reverted the change before enough traffic hit the clone, a direct product of having the detection and response steps below already rehearsed. That contrast, one protocol confirmed compromised and phishing live, two others recovering with no confirmed loss, is the entire argument for treating this chapter as a rehearsed playbook rather than an improvised response.

A defacement or compromise can originate from any of the layers in the Section 15.2 table: a hosting account, a CDN account, a registrar, or a build pipeline. The response playbook below is deliberately layer-agnostic in its first three steps, because from the outside a user cannot tell which layer failed, and neither, usually, can the team in the first ten minutes.

Detection signals (any one should trigger the playbook; two or more should trigger full incident status):

Signal	Source	What it suggests
User reports of a different page, wallet drainer, or unexpected approval prompt	Support channels, Discord, X/ Twitter mentions	Content substitution somewhere upstream of the browser
DNS record mismatch against your known-good baseline	Automated DNS monitoring (Part 2, Section 3)	Registrar or DNS-layer takeover
Unrecognized deployment or file hash on the hosting platform	Hosting platform audit log, deployment history	Hosting or CI/CD account compromise
CSP violation reports spiking for an unfamiliar script origin	CSP `report-to` endpoint (Handbook 01)	Injected script, possibly CDN-layer
TLS certificate transparency log entry you did not request	crt.sh or CT monitoring feed	New certificate issued for your domain, often preceding a takeover going live

Response steps:

1. **Confirm before acting.** Load the site from a clean, uncached network path (a fresh browser profile or a mobile connection) and diff it against a known-good snapshot. Compare the resolved IP or CNAME target and the TLS certificate's issuer and serial number against your baseline. False alarms from a stale local cache are common; do not lock out legitimate operators over one.
2. **Contain at the layer that failed.** If DNS or the registrar is compromised, follow the Part 2 registrar-compromise playbook to regain account control and revert records. If the hosting platform shows an unauthorized deployment, revoke the platform's active sessions and API tokens, and roll back to the last known-good immutable deployment rather than attempting to patch the live one. If a CDN account or edge script is implicated, follow Handbook 01's guidance and disable the offending Worker, function, or edge rule.
3. **Warn users immediately, even before root cause is confirmed.** Post to every channel you control that you do not control (a pinned tweet, a status page, a Discord announcement) that the primary domain may be compromised and instruct users not to connect wallets or sign anything until you confirm it is safe. Speed here matters more than completeness; an early, corrected warning beats a late, precise one measured in drained wallets.
4. **Preserve evidence before remediating destructively.** Export the hosting platform's audit log, the DNS zone history, the CDN account's activity log, and, if you can capture it safely, a copy of the malicious content itself (its script, its destination addresses, its drainer kit signature) before you delete or overwrite anything. This evidence feeds both your own postmortem and any law-enforcement or ecosystem-wide (Scam Sniffer, SlowMist, Chainabuse) reporting.
5. **Recover onto verified infrastructure, not merely reverted infrastructure.** Rotate every credential in the chain that touched the compromised layer, not only the one directly implicated: registrar password and MFA method, hosting platform tokens, CI/CD secrets, and any team member accounts that had access. Re-deploy from a known-good, signed

build artifact rather than trusting the currently-live one, even after DNS is reverted, since a hosting-layer compromise can persist after a DNS-layer fix.

6. **Publish a postmortem.** Follow the transparency norm set by incidents like BadgerDAO and the Squarespace-affected protocols: state the timeline, the root cause, the funds at risk versus funds lost, and the concrete control added to prevent recurrence. A postmortem that stops at "we fixed it" without naming the mechanism does not help the next team read this handbook after their own incident.

---

17. Integration with Incident Response

This Part's playbook is a specialization, not a substitute, for the general incident-response process a mature organization already runs. NIST SP 800-61 Revision 3, "Incident Response Recommendations and Considerations for Cybersecurity Risk Management" (April 2025), retired the older, purely linear four-phase model (Preparation, Detection and Analysis, Containment/Eradication/Recovery, Post-Incident Activity) in favor of organizing incident response around the six functions of the NIST Cybersecurity Framework (CSF) 2.0: Govern, Identify, Protect, Detect, Respond, and Recover. The change matters for this handbook because it explicitly folds incident response into ongoing risk management rather than treating it as a separate emergency process that only starts once something is already on fire, which is exactly the posture Section 15's hardening controls and Section 16's rehearsed playbook are meant to support before an incident, not only during one.

Map the front-end defacement playbook onto those functions directly. **Govern** is the decision, made now, that this handbook's Section 16 steps are the organization's approved procedure for this incident class, with a named owner for each step. **Identify** is the DNS inventory and hosting-account map from Section 15.1: you cannot contain a compromised layer you have not enumerated. **Protect** is the hardware-key MFA, least-privilege tokens, and immutable-deployment practices from Sections 15.1 and 15.2. **Detect** is the monitoring signals table in Section 16. **Respond** is Section 16's numbered steps 1 through 5. **Recover** is step 6, the postmortem, plus the credential rotation and verified re-deployment that close the loop back into Protect.

The Incident Response Handbook (06) owns the parts of this lifecycle that are common to every incident class in the series regardless of which layer failed: legal and regulatory notification obligations, internal and external communication cadence and approval chains, chain-of-custody standards for evidence you plan to hand to an exchange, a law-enforcement contact, or an on-chain forensics firm, and the structured post-incident review format. This handbook's playbooks, here and in Parts 2 through 4, supply the domain-, resolver-, gateway-, and hosting-specific detection signals and containment actions that Handbook 06's general process treats as inputs. Treat the relationship as a plug-in: when Section 16's playbook reaches step 3 (warn users) or step 6 (postmortem), that is the exact point where you hand off to Handbook 06's communication

and review procedures rather than improvising a second, competing process. A team that runs both processes independently, one from this handbook and one from Handbook 06, without an explicit handoff point tends to duplicate effort during the ten minutes that matter most and skip the post-incident review once the immediate fire is out.

Key controls for Part 5

- Enforce hardware-key MFA on every account that can change what a hosting, PaaS, or CDN platform serves; treat that login the same as a signing key.
- Scope deploy and API tokens to a single project with least privilege, and store them per the OWASP Secrets Management Cheat Sheet, never inline in a repository or workflow file.
- Maintain a live inventory of every DNS record and the cloud resource it points to, and remove the record the same day a resource is deprovisioned to close the subdomain-takeover window.
- Deploy from immutable, versioned artifacts so rollback is redeploying a known-good build, not racing a mutable pointer.
- Know which handbook governs which layer (DNS, hosting, CDN, build, broader infrastructure) before an incident, so response time is not spent figuring out where to look.
- Rehearse the Section 16 playbook and its Section 17 handoff into Handbook 06 before you need it; the Squarespace-era protocols that recovered without confirmed loss did so because detection and reversion were already fast, not because the underlying registrar flaw was any less severe for them.

Part VI: Appendices

This part collects the reference material the rest of the handbook assumes you have on hand. Appendix A fixes the vocabulary so *contenthash* or *registry lock* means the same thing in every Part. Appendix B turns the hardening guidance from Parts 2 through 5 into copy-ready checklists and a risk-register template for a deployment runbook. Appendix C is the one-page router Part 1 promised: a table that sends a responder to the right playbook before they know which naming system failed. Appendix D points to the handbook's bibliography rather than repeating citations already given at the point of each claim.

18. Appendix A: Glossary

Entries are grouped by domain area, not strict alphabetical order across the whole appendix, because a responder triaging a registrar compromise and one debugging a stale IPFS pin reach for different clusters of vocabulary. Within each group, entries run alphabetically, with the acronym expanded on first appearance and the handbook chapter named so you can jump to the fuller treatment. Where an external standard, request for comments (RFC), or Ethereum Improvement Proposal (EIP) defines a term precisely, the entry cites that source directly.

18.1 DNS and Routing Terms

The classical resolution and routing path covered in Part 2.

- **Authoritative nameserver**: the server holding a domain's definitive zone file (its full set of DNS resource records) and answering queries for it directly, as opposed to a caching resolver that only forwards and caches answers on a client's behalf. See Chapter 2.
- **BGP (Border Gateway Protocol) hijack**: an attacker announces false route prefixes to internet peers, misdirecting traffic destined for a victim's IP space upstream of any control the victim can apply to its own domain or server. Documented against MyEtherWallet (April 2018) and KLAYswap (February 2022); see Chapter 1.2.
- **CAA (Certification Authority Authorization)**: a DNS resource record, defined in RFC 8659, letting a domain holder restrict which certificate authorities may issue TLS

certificates for that name, using `issue`, `issuewild`, and `iodef` property tags. See Chapter 3.3.

- **Cache poisoning (DNS)**: injecting a forged resource record into a resolver's cache so subsequent queries return an attacker-controlled answer, with no registrar or registry access required. See Chapter 2.1.
- **DNS hijacking**: any technique causing a domain to resolve to attacker-controlled infrastructure, spanning registrar account takeover, unauthorized nameserver delegation changes, and cache poisoning. Curve Finance lost more than $570,000 to a registrar-level hijack in August 2022; see Chapter 1.2.
- **DNSSEC (Domain Name System Security Extensions)**: IETF extensions, originating in RFC 4033 through RFC 4035, letting a resolver cryptographically verify a DNS answer came from the zone's legitimate holder and was not altered in transit. See Chapter 3.1.
- **DNSKEY**: the DNSSEC record publishing a zone's public signing key, verified against its parent via the DS record. Defined in RFC 4034. See Chapter 3.1.
- **DS (Delegation Signer)**: a DNSSEC record in the parent zone containing a hash of the child zone's DNSKEY, forming the cryptographic link in the chain of trust from the root to a leaf domain. Defined in RFC 4034. See Chapter 3.1.
- **EPP (Extensible Provisioning Protocol) status codes**: machine-readable domain flags, defined in RFC 5731, including `clientTransferProhibited`, `clientUpdateProhibited`, and `clientDeleteProhibited`, that instruct a registry to reject the matching operation until the registrar removes the flag; the mechanism behind registrar lock. See Chapter 3.3.
- **Registrar lock**: setting the client-side EPP prohibition codes on a domain so transfers, updates, or deletion require an explicit unlock step, closing the most common path to a fast, silent hijack. See Chapter 3.3.
- **Registry lock**: a stronger protection some registries offer for high-value domains, requiring an out-of-band step (typically a phone callback to a pre-authorized contact) before any change takes effect, even one authenticated through the normal account. See Chapter 3.3.
- **Resolver validation**: a DNS resolver actually checking DNSSEC signatures on the answers it returns, rather than merely being capable of it. Validating and non-validating resolvers coexist; global validation share is tracked at APNIC Labs' DNSSEC measurement. See Chapter 3.4.
- **RRSIG (Resource Record Signature)**: the DNSSEC record holding the digital signature over a set of resource records, verified against the corresponding DNSKEY. Defined in RFC 4034. See Chapter 3.1.
- **WHOIS**: the protocol and data set for domain registration contact information, used to identify a registrant and, defensively, to monitor for unauthorized registrant or nameserver changes. See Chapter 3.5.

18.2 ENS and Ethereum Naming Terms

The on-chain naming layer covered in Part 3.

- **contenthash**: an ENS resolver record, defined in EIP-1577, letting a name such as `app.eth` resolve to an IPFS, InterPlanetary Name System (IPNS), or Swarm content identifier instead of, or alongside, an Ethereum address. See Chapter 7.1 and Part 4, Chapter 12.1.
- **CCIP Read (Cross-Chain Interoperability Protocol Read)**: a standard, defined in EIP-3668, letting a smart contract instruct a client to fetch data from an off-chain gateway and return it as a verified on-chain call; the mechanism ENS uses to verify DNSSEC proofs without paying gas for every lookup. See Chapter 7.2.
- **DNSRegistrar / DNSSECOracle**: the ENS smart contracts behind DNS-in-ENS. `DNSRegistrar` historically accepted an on-chain DNSSEC proof so a DNS holder could claim the matching ENS name, at significant gas cost; `DNSSECOracle` verifies that proof chain for both the on-chain path and the newer gasless, CCIP Read-based path. See ENS documentation and Chapter 7.2.
- **ExtendedDNSResolver**: an ENS resolver that reads a TXT record on an imported DNS name and resolves it to an address directly, without the name being claimed on-chain first, part of the gasless DNS-in-ENS flow. See Chapter 7.2.
- **namehash**: the deterministic algorithm, defined in EIP-137, converting a human-readable ENS name like `wallet.app.eth` into the fixed-length node identifier ENS contracts store and compare. See Chapter 7.1.
- **Resolver (ENS)**: the smart contract an ENS name points to that answers what the name resolves to (address, `contenthash`, text record). Because the registry only records which resolver a name uses, a compromised resolver can misdirect every lookup without touching name ownership. See Chapter 7.4.
- **Subdomain takeover (ENS)**: gaining control of a subdomain, typically through a wildcard resolver answering for any label under a parent name without per-label checks, letting an attacker resolve `evil.app.eth` with the apparent legitimacy of a name the project actually issued. See Chapter 8.1.

18.3 IPFS and Decentralized Hosting Terms

Content-addressed storage and retrieval, covered in Part 4.

- **CID (Content Identifier)**: a self-describing label for IPFS content, built from a multihash (cryptographic digest), a multicodec (how to interpret the bytes), and a multibase (text encoding), such that any content change changes the CID. See IPFS docs and Part 4, Chapter 11.1.
- **CIDv0 / CIDv1**: the two CID encoding versions in use. CIDv0 is a fixed Base58 string starting with `Qm`; CIDv1 adds an explicit multibase/multicodec prefix and is required for

subdomain gateway hosting because it can be a DNS-label-safe string. See Part 4, Chapter 11.1.

- **DHT (Distributed Hash Table)**: the Kademlia-style peer-to-peer index IPFS nodes use to discover which peers hold a given CID's blocks. See Part 4, Chapter 12.2.
- **DNSLink**: a convention for publishing a `_dnslink.` prefixed TXT record that maps a domain name to an IPFS path, letting a human-readable URL resolve to a CID the same way an ENS `contenthash` does. See Part 4, Chapter 12.1.
- **Gateway (IPFS)**: an HTTP bridge process speaking IPFS on one side and plain HTTPS on the other, used by clients that do not run a full IPFS node. Path, subdomain, and DNSLink gateway styles differ in browser origin isolation. See IPFS HTTP Gateway spec and Part 4, Chapter 11.2.
- **IPNS (InterPlanetary Name System)**: IPFS's mutable-pointer scheme, letting a stable identifier resolve to a changing CID over time; one of the target types an ENS `contenthash` record can point to. See Part 4, Chapter 12.1.
- **Pinning**: explicitly instructing an IPFS node to retain content indefinitely, preventing garbage collection; unpinned content can silently vanish once the last holding node goes offline. See IPFS docs and Part 4, Chapter 11.2.
- **Storacha Network**: the successor to Protocol Labs' web3.storage pinning service, built on the `w3up` protocol stack and backed by Filecoin storage deals. See github.com/storacha and Part 4, Chapter 12.2.
- **Sybil / eclipse attack**: a Sybil attack floods a peer-to-peer routing table (such as a DHT's) with attacker-controlled identities; an eclipse attack uses that foothold to surround one target peer with malicious nodes, isolating it from honest ones. Both are documented structural exposures of Kademlia-family DHTs. See Part 4, Chapter 12.2.
- **Trustless (CAR-verified) retrieval**: requesting an IPFS gateway response in CAR format so the client hashes the raw DAG blocks itself, detecting a dishonest gateway instead of trusting its word for what a CID contains. See Part 4, Chapter 11.2.

18.4 Hosting, Deployment, and Incident Response Terms

Conventional hosting and the response process, covered in Part 5 and cross-referenced throughout the playbooks.

- **Defacement**: unauthorized modification of a front end's visible content or behavior, from cosmetic vandalism to an injected malicious wallet-signing prompt, without necessarily touching DNS, ENS, or IPFS resolution. See Chapter 16.
- **NIST incident handling lifecycle**: the four-phase model (Preparation; Detection and Analysis; Containment, Eradication, and Recovery; Post-Incident Activity) defined in NIST SP 800-61 Revision 2 (August 2012), which every playbook in this handbook adapts to a specific naming or hosting failure. See Chapter 17.

- **SRI (Subresource Integrity)**: a browser mechanism pinning the cryptographic hash of an external script or stylesheet, rejecting a resource whose fetched bytes mismatch; the primary content-integrity control in the CDN and Front-End Supply Chain Security Handbook (01), referenced here where hosting and CDN concerns overlap. See Chapter 15.2.
- **VPS (Virtual Private Server)**: a virtualized server rented from a cloud or hosting provider, the common origin infrastructure a domain's A or AAAA records ultimately point to. See Chapter 15.1.

18.5 Standards, Frameworks, and Governance Terms

The bodies and frameworks that structure this handbook's guidance within the wider OWASP Smart Contract Security (SCS) Project.

- **AADAPT (Adversarial Actions in Digital Asset Payment Technologies)**: a MITRE threat framework modeled on ATT&CK, launched in 2025, cataloging adversary tactics specific to digital asset and payment systems, including infrastructure and naming-layer attacks. See MITRE's fact sheet and the public matrix.
- **ICANN (Internet Corporation for Assigned Names and Numbers)**: the nonprofit body coordinating the DNS root zone, accrediting registrars, and setting the policy framework (EPP status codes, transfer rules) this handbook's registrar-security guidance builds on. See icann.org.
- **MITRE ATT&CK**: a globally accessible knowledge base of adversary tactics and techniques drawn from real-world observations, used to structure general IT and cloud threat models that intersect with DNS and hosting infrastructure. See attack.mitre.org.
- **NIST (National Institute of Standards and Technology)**: the U.S. federal agency whose Special Publication 800 series, including SP 800-81-2 and SP 800-61 Revision 2, anchors this handbook's DNS hardening and incident-response structure.
- **SCSVS / SCSTG / SCWE**: OWASP's Smart Contract Security Verification Standard, Testing Guide, and Weakness Enumeration; this handbook sits outside all three by design, supplying the DNS, ENS, and IPFS preconditions they assume are already met. See scs.owasp.org and Chapter 1.3.
- **SSAC (Security and Stability Advisory Committee)**: an ICANN advisory committee publishing analysis and recommendations on DNS security matters, including registrar-level abuse and hijacking prevention, referenced throughout Part 2's hardening guidance. See icann.org/groups/ssac.
- **Web3 Attack Vectors Top 15**: an OWASP catalog ranking the most significant non-contract Web3 risks; it names DNS, domain, and routing infrastructure hijacking directly as **WA13**, the category this handbook is organized around. See scs.owasp.org/sctop10/Web3-Attack-Vectors-Top15 and Chapter 1.2.

19. Appendix B: DNS/ENS/IPFS Security Checklist

The checklists below are the copy-ready form of the hardening guidance argued for in prose across Parts 2 through 5. Lift a section wholesale into a pre-launch review or a change ticket rather than reconstructing it from memory. Most teams run all four checklists together at least once per quarter, since a domain, an ENS name, an IPFS pin, and a hosting origin typically ship as one release.

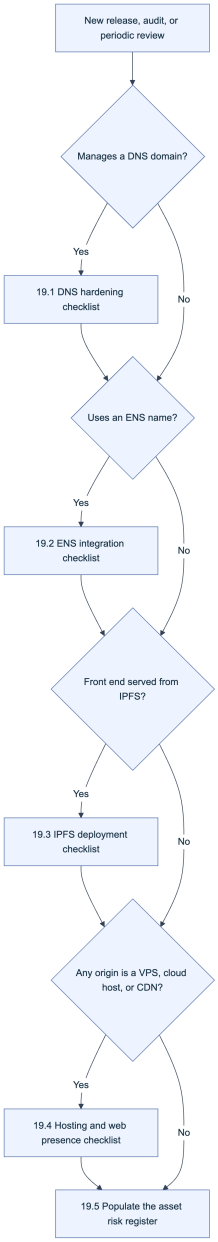

Figure 1. Checklist selection by system in use. Most production Web3 front ends run at least three of the four branches simultaneously, since DNS, ENS, and IPFS are commonly layered rather than mutually exclusive.

19.1 DNS Hardening Checklist

Run this checklist before launch and again on a fixed schedule (monthly is typical) independent of releases, since registrar and DNS configuration can drift without any deployment on your side. It operationalizes Chapter 3.

DNS Hardening Checklist

- ☐ Zone is DNSSEC-signed; DS/DNSKEY/RRSIG chain validates end to end (`dig +dnssec`, cross-check with DNSViz)
- ☐ CAA record restricts issuance to chosen CAs, includes `iodef` tag
- ☐ Registrar lock set: clientTransferProhibited, clientUpdateProhibited, clientDeleteProhibited
- ☐ Registry lock enabled where offered, for high-impact domains
- ☐ Hardware-backed MFA enforced on every registrar account; no shared or generic logins
- ☐ WHOIS contact info current, monitored for unauthorized change
- ☐ Expiry tracked with auto-renewal plus 60/30/7-day alerts
- ☐ Nameserver and zone changes alert independent of the registrar's own notification (Chapter 3.5)
- ☐ Certificate issuance monitored via Certificate Transparency logs
- ☐ No dangling records point at decommissioned infrastructure a third party could re-claim

19.2 ENS Integration Checklist

Run this checklist whenever an ENS name is provisioned or its resolver, controlling key, or records change. It operationalizes Chapters 7, 8, and 10.

ENS Integration Checklist

- ☐ Name-controlling key is a hardware wallet or multisig/timelock, never a bare hot-key externally owned account
- ☐ Resolver contract is a known, reviewed implementation, not an arbitrary or freely-upgradeable address
- ☐ contenthash and address record writes are monitored and alerted on
- ☐ Wildcard resolution disabled unless subdomains are explicitly issued, closing the common subdomain-takeover path
- ☐ DNS-imported names (DNSSECOracle/CCIP Read path) re-verified after any parent DNS zone change (Chapter 7.2)
- ☐ Front end displays the full resolved name and flags homoglyph or lookalike labels before wallet connection
- ☐ Referral/forwarding links are validated before rendering, never trusted from an unauthenticated query parameter (Chapter 10.2)

19.3 IPFS Deployment Checklist

Run this checklist before pinning a new build and again on a fixed schedule. It operationalizes Part 4, Chapter 14.

IPFS Deployment Checklist

- ☐ Content pinned to two or more independent pinning providers
- ☐ Sensitive content retrieved via trustless CAR responses and hash-verified locally, not trusted from a recursive gateway
- ☐ Anything executing code is served as CIDv1 via a subdomain gateway, never a path-style gateway
- ☐ contenthash/DNSLink owner key is hardware-backed and, where possible, multisig-protected
- ☐ Deployed CID is polled continuously and alerted against the release manifest
- ☐ Canonical CID published out of band: GitHub, ENS text record, verified social channels
- ☐ Dedicated or self-hosted gateway fallback exists for users behind networks that block public gateway domains
- ☐ Critical pins backed by a direct storage relationship (a held Filecoin deal or equivalent), not solely a reseller account

19.4 Hosting and Web Presence Checklist

Run this checklist for every origin server, VPS, or cloud host a domain record points to. It operationalizes Chapter 15 and its overlap with the CDN and Front-End Supply Chain Security Handbook (01).

Hosting and Web Presence Checklist

- ☐ Origin is not directly reachable bypassing the CDN; access is authenticated pull, IP allowlisting, or mutual TLS only
- ☐ TLS 1.3 enforced (1.2 minimum) with HSTS enabled
- ☐ Deployment pipeline produces signed, versioned, immutable build artifacts (Handbook 01, Chapter 4.3-4.4)
- ☐ SRI and CSP applied to every third-party script and style (Handbook 01, Chapter 4)
- ☐ Server software is current and minimal: no exposed default admin panels, no unused listening services
- ☐ Tested rollback path exists to redeploy a known-good build within a defined recovery time objective
- ☐ Secondary web presence (docs, status pages, blogs) is inventoried in the same monitoring scope as the primary application domain

19.5 Domain, Name, and Content Risk Register Template

Maintain this register as a living document across all four systems rather than four separate spreadsheets; a single asset row makes it obvious when, for example, an ENS name and its DNS-imported counterpart drift out of sync.

Asset	Type (Domain / ENS name / IPFS pin / Host)	Controlling Party or Key	Lock/Protection Status	Monitoring in Place	Last Reviewed	Owner
(example row)	Domain (`app.example`)	Registrar account, hardware MFA	Registrar lock + registry lock	Uptime + NS-change alert	2026-Q3	Infra lead
(example row)	ENS name (`app.eth`)	3-of-5 multisig	Resolver reviewed, wildcard off	contenthash write alert	2026-Q3	Protocol lead
(your asset)						

20. Appendix C: Playbook Quick Reference

An incident responder rarely knows which naming system failed in the first few minutes; the symptom, users landing on a page that is not the real front end, looks identical whether the cause is a hijacked registrar, a compromised ENS resolver, a substituted IPFS CID, or a defaced origin server. This appendix is the one-page router Chapter 1.4 promised: scope the failure first, then jump to the matching playbook rather than reading each Part's full threat model under pressure.

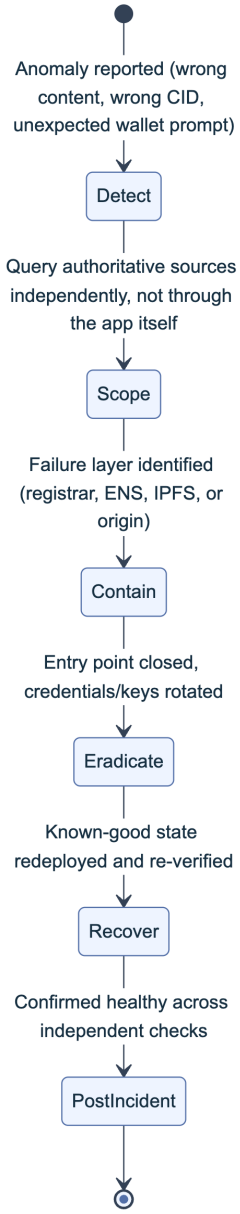

Figure 2. The generalized incident lifecycle every playbook in this handbook specializes, adapted from the four-phase model in NIST SP 800-61 Revision 2. Scoping happens before containment in every case, because the correct containment action differs completely depending on which layer actually failed.

20.1 Playbook Index and Trigger Table

Use the trigger column to route before you have a confirmed root cause; symptoms overlap heavily by design, which is exactly why an out-of-band scoping step (right-hand column) comes first in every playbook.

Playbook	Location	Trigger Symptom	First Independent Verification Step
Suspected DNS Hijack or Poisoning	Part 2, Ch. 4	Front end resolves to unexpected content; DNS changed without a matching ticket	Query authoritative nameservers from an out-of-band resolver; do not trust the app's own state
Domain Expiry or Registrar Compromise	Part 2, Ch. 5	WHOIS shows expired/pending-delete, or registrar account access is lost	Attempt registrar-side recovery immediately; check backup MFA before the redemption grace period lapses
DNS Abuse (Phishing, Malware on Your Brand)	Part 2, Ch. 6	Reports of a phishing clone or malware host on a subdomain or lookalike using your brand	Confirm the abusive host is not, in fact, your own hijacked infrastructure before filing a takedown
ENS Name or Resolver Compromise	Part 3, Ch. 9	contenthash/address record changed unexpectedly, or key suspected compromised	Query the resolver against a trusted RPC (`cast call`), never through the potentially-compromised front end
IPFS Gateway or Pinning Incident	Part 4, Ch. 13	Served CID mismatches the release manifest, or a gateway returns stale content	Test the same CID across 3+ independent gateways to isolate pointer-layer vs. gateway fault
Front-End Defacement or Compromise	Part 5, Ch. 16	Visible content change, injected script, or unexpected wallet-signing prompt reports	Fetch the origin out-of-band (`curl` from an unrelated host) and diff against the last known-good build

20.2 Universal First 15 Minutes Checklist

These actions apply regardless of which playbook you end up running, and belong at the top of every incident channel the moment a naming or hosting anomaly is reported.

Universal First 15 Minutes

- ☐ Preserve evidence first: screenshot the anomaly, save raw `dig` / `whois` / `cast call` output with timestamps
- ☐ Verify independently, never through the potentially-compromised front end or its own status page
- ☐ Do not tip off an active attacker with public changes before containment is ready; coordinate the first public statement
- ☐ Identify the failed layer (DNS, ENS, IPFS, origin) before choosing containment; the wrong step can be inert or harmful (Chapter 1.4)
- ☐ Open the incident under the Incident Response Handbook (06) process and pull in the specific playbook from 20.1

20.3 Escalation and Communication Matrix

Different failure layers require contacting different external parties, and the wrong contact wastes the first, most valuable minutes of an incident. Populate the specific account and contact details for your own registrar, ENS tooling, and pinning providers in advance, not during the incident.

Failure Layer	External Party to Contact	Typical Channel	What to Provide
Registrar / registry compromise	Registrar abuse/security team, then registry	Support escalation line, ICANN complaint form	Domain name, account ID, evidence of change, timestamp
ENS resolver or key compromise	ENS support channels, multisig co-signers	Community support, internal signer escalation	Name, unauthorized tx hash, current resolver address
IPFS gateway or pinning	Pinning provider support, gateway abuse contact	Support ticket, gateway abuse email	CID, last known-good CID, pin status screenshots
Origin hosting or CDN	Hosting/cloud provider security team	Provider incident escalation line	Affected IPs, timeline, evidence of access
Users and community	Verified status page, verified social account	Pre-established, out-of-band channels only	Plain-language impact statement, what to avoid (e.g. signing)
Law enforcement / recovery	Local cybercrime unit, analytics partner	Formal report, per Handbook 06	Evidence package, loss estimate, wallet addresses

21. Appendix D: References and Further Reading

Every claim in this handbook that names a standard, a version, a date, or an incident is cited inline at the point it is made, following the convention set in Part 1. Rather than duplicate that source list here, this appendix points to `references.md`, the deduplicated bibliography for the full handbook, grouped into Standards and Frameworks (the DNSSEC RFC series, RFC 8659 for CAA, RFC 5731 for EPP status codes, EIP-1577 and EIP-3668 for ENS `contenthash` and CCIP Read, NIST SP 800-81-2 and SP 800-61 Revision 2, and the OWASP SCSVS, SCSTG, SCWE, and Web3 Attack Vectors Top 15), Incidents and Case Studies (MyEtherWallet's 2018 BGP hijack, KLAYswap's 2022 BGP-hijacked SDK compromise, and the Curve Finance 2022 registrar compromise), Tools (`dig`, DNSViz, `cast`, Kubo, and the pinning provider APIs referenced across Part 4), and Further Reading (ICANN and SSAC advisories, IPFS specification documents, and companion material beyond this handbook's scope).

If a link in the body of any Part goes stale, `references.md` is the place to check for an updated URL before assuming the underlying claim no longer holds. When you extend this handbook, add the new source to `references.md` in the same change, so the bibliography stays a complete index.

Key controls for Part 6

- Treat Appendix A as the canonical vocabulary across DNS, ENS, IPFS, and hosting; link to it from tickets and post-mortems instead of redefining a term inline each time.
- Run the system-specific checklists (19.1 through 19.4) before every launch and on a fixed periodic schedule, not only after an incident prompts one.
- Keep the asset risk register (19.5) as a single living document spanning all four systems, so drift between an ENS name and its DNS-imported counterpart is visible in one place.
- Use the playbook quick reference (20.1) to scope an incident by symptom before committing to a containment action; the wrong layer's fix can be inert or harmful.
- Populate the escalation matrix (20.3) with real contacts before an incident, not during one.
- Add every new citation to `references.md` in the same change that introduces it (Appendix D).

Appendix C: References and Further Reading

This appendix collects every external source cited across the handbook's parts, grouped by category for easy reference.

Standards and Frameworks

- RFC 1035, Domain Names - Implementation and Specification
- RFC 4033, DNS Security Introduction and Requirements
- RFC 4034, Resource Records for the DNS Security Extensions
- RFC 4035, Protocol Modifications for the DNS Security Extensions
- RFC 5731, Extensible Provisioning Protocol (EPP) Domain Name Mapping
- RFC 8659, DNS Certification Authority Authorization (CAA) Resource Record
- RFC 9364 (BCP 237), DNS Security Extensions (DNSSEC)
- NIST SP 800-81-2, Secure Domain Name System (DNS) Deployment Guide
- NIST SP 800-61 Revision 2, Computer Security Incident Handling Guide
- NIST SP 800-61 Revision 3, Incident Response Recommendations and Considerations for Cybersecurity Risk Management
- NIST Cybersecurity Framework (CSF) 2.0
- ICANN
- ICANN Security and Stability Advisory Committee (SSAC)
- ICANN SAC115, SSAC Advisory on DNS Abuse
- OWASP Secrets Management Cheat Sheet
- OWASP Smart Contract Security (SCS) Project
- OWASP Web3 Attack Vectors Top 15
- MITRE ATT&CK
- MITRE ATT&CK T1078.004, Valid Accounts: Cloud Accounts
- MITRE ATT&CK T1584.002, Compromise Infrastructure: DNS Server
- MITRE AADAPT Cyber Threat Framework for Digital Assets, fact sheet
- MITRE AADAPT public matrix (GitHub)
- CIS Benchmarks
- AWS Well-Architected Framework, Security Pillar
- EIP-137, Ethereum Domain Name Service - Specification
- EIP-1577, contenthash field for ENS
- EIP-3668, CCIP Read: Secure offchain data retrieval
- ENSIP-8, Interface Discovery
- ENSIP-9, Multichain Address Resolution

- ENSIP-11, EVM-Compatible Chain Address Resolution
- ENSIP-15, ENS Name Normalization Standard
- ENSIP-17, Gasless DNS Resolution
- ENSIP-19, Multichain Primary Names
- IPFS HTTP Gateway Specification
- IPFS Trustless Gateway Specification (CAR format)
- Multiformats

Incidents and Case Studies

- Cisco Talos, Sea Turtle DNS hijacking campaign
- Cisco Talos, sustained abuse of IPFS gateways
- CyberScoop, Ether DNS/BGP Amazon Route 53 heist
- Internet Society, “What Happened? The Amazon Route 53 BGP Hijack”
- The Record, KlaySwap crypto users lose funds after BGP hijack
- CertiK, BGP Hijacking: the $1.9M KlaySwap Attack Through Manipulated Network Flow
- Decrypt, Curve Finance DNS Record Attack
- CryptoTimes, Curve Finance Loses $570K Due to DNS Compromise
- rekt.news, Curve Finance
- The Register, MyEtherWallet DNS Hijack
- Namefi, “The 2024 Squarespace DeFi Domain Mass-Hijack”
- SC Media coverage via dnsafrica.org, Squarespace-Registered DeFi Platforms Subjected to DNS Hijacking
- Smart Contract Audit, BadgerDAO hack (2021)

Tools and Documentation

- crt.sh, Certificate Transparency search
- `can-i-take-over-xyz` project (GitHub)
- Storacha Network (GitHub)
- ENS Docs: Architecture
- ENS Docs: DNS Registry
- ENS Docs: Universal Resolver
- ENS Docs: Name Wrapper
- ENS blog, How ENS Normalization Works
- Basenames
- IPFS docs: content addressing
- IPFS docs: IPFS gateway
- IPFS docs: nodes and network layer

- IPFS docs: persistence, pinning, and garbage collection
- APNIC Labs, DNSSEC validation measurement
- APNIC blog, How We Measure DNSSEC Validation

Further Reading

- Wikipedia, BGP hijacking
- Wikipedia, DNS spoofing
- Wikipedia, DNSSEC
- Martin Fowler / Kief Morris, Immutable Server pattern